

Deep-Reinforcement-Learning-Based Single-Input Pinning Synchronization of Hopfield Chaotic Neural Networks with an Application to Image Encryption

Yan'an Huang^{a,1}, Zixuan Qiao^{a,1}, Jiabing Zhu^b, Weiye Wang^a, Xiaoyan Sun^a, Chenchen Yu^c, Jie Wu^{a,*} and Ru-ru Ma^{d,*}

^aSchool of Artificial Intelligence and Computer Science, Jiangnan University, Wuxi, 214122, China

^bInspur Software Group Co., Ltd, Jinan, 250000, China

^cQingdao Haier Air Conditioner Electronic Co., Ltd, Qingdao, 266101, China

^dSchool of Mathematics and Physics, Suzhou University of Science and Technology, Suzhou, 215009, China

ARTICLE INFO

Keywords:

Image encryption

Synchronization

Deep reinforcement learning

Neural network

ABSTRACT

A single control input limits where feedback enters a chaotic system, but leaves open which variables the controller should observe. This study investigates how these two design choices interact in the synchronization of a continuous-time Hopfield network. Proximal policy optimization (PPO) is used to learn bounded feedback, with local node-screening indicators and a state-boundedness analysis guiding the assessment. The central finding is that observation composition can matter as much as input dimension: changing the coordinate subset improves synchronization at Node 2 but worsens it at Node 3, despite retaining the same observation size. A targeted soft actor-critic (SAC) comparison reproduces this direction of change, while revealing algorithm-dependent differences in control effort. An accuracy-constrained screening example translates the observed preferences into choices within a fixed candidate pool. Comparisons across all actuator subsets further show the cost of restricting control to one node: favorable multi-input configurations improve full-horizon accuracy at greater total effort, whereas steady-state precision follows a different ordering. Traditional-controller and SAC benchmarks place these findings in the context of controller-dependent accuracy–effort tradeoffs. An image encryption and decryption demonstration provides an application of the synchronized trajectories. Together, the results show why actuator placement, feedback information, and control effort should be considered jointly when designing single-input synchronization for the studied network.

1. Introduction

Chaotic synchronization concerns the coordination of sensitive nonlinear trajectories through coupling or feedback, with applications in secure communication and coordinated dynamical systems [1–4]. In a drive–response architecture, the control must make a response system follow an independently evolving drive. When only one actuator is available, a scalar input must influence all synchronization-error components through the internal coupling. The actuator location therefore becomes part of the control design: a controller that performs well at one coordinate may require different feedback information or control effort at another.

Linear feedback, adaptive control, sliding-mode control (SMC), and pinning strategies provide established approaches to synchronization [5–9]. These methods supply analytical structure and, under their respective assumptions, stability results. Deep reinforcement learning (DRL) offers a complementary route by fitting nonlinear feedback from observations and rewards [10–12]. Applications to chaos

control and synchronization have demonstrated the feasibility of this approach across different dynamical systems [13–18]. This motivates examining what learned feedback can achieve under a prescribed actuator constraint, while retaining model-based controllers as informative references.

Observation availability is a separate design variable. A single actuator does not require that the controller observe only the actuated coordinate: sensing determines the available information, whereas actuation determines where feedback enters the dynamics. Recent work has examined reinforcement learning with reduced inputs and incomplete observations [19, 20]. In particular, Weissenbacher et al. [20] study reduced sensing and memory architectures in chaotic-flow control. For drive–response synchronization, reduced sensing must be considered together with the scalar actuator through which the policy influences the response. Ozan et al. [21] also examine sensor and actuator locations in partially observed chaotic-flow control. Building on this sensor–actuator perspective, the present study focuses on the consequences of observation composition for single-coordinate drive–response synchronization in the specified Hopfield network.

Learned synchronization with incomplete state information has been demonstrated for identical and distinct chaotic systems [15, 16]. This raises a more specific design question:

*Corresponding authors. E-mail addresses: jiewu20@jiangnan.edu.cn (J. Wu); maruru7271@126.com (R. Ma).

6253111028@stu.jiangnan.edu.cn (Y. Huang);
6253114027@stu.jiangnan.edu.cn (Z. Qiao); zhujb@inspur.com (J. Zhu);
6243115018@stu.jiangnan.edu.cn (W. Wang); xysun78@126.com (X. Sun);
m15866813396@163.com (C. Yu)

ORCID(s):

¹These authors contributed equally to this work.

when feedback information is restricted, how does its usefulness depend on the node receiving the control input? Related soft-computing approaches include critic-based neuro-fuzzy chaos control [22], Lyapunov-analyzed adaptive fuzzy synchronization [23], and Lyapunov-based reward shaping for deep reinforcement learning in tracking control [24]. These methods motivate distinguishing learning performance from guarantees established for a particular controller structure.

The continuous-time Hopfield-type network provides a compact setting for this investigation. Unlike the symmetric network associated with equilibrium-seeking dynamics [25], asymmetric Hopfield-type connections can produce chaotic behavior [26–28]. Existing work includes dynamical analysis and synchronization using model-based, impulsive, intermittent, and event-triggered control [29–33]. The three-state model considered here permits examination of every actuator subset and direct comparison of local dynamical indicators with nonlinear closed-loop behavior. Its role is to make the constrained-control design choices inspectable; extension to larger networks requires separate evaluation.

These considerations lead to two connected questions. First, which observation variables are useful at a given actuator location? Second, what accuracy and control-effort changes accompany the addition of control channels? We address both using a common proximal policy optimization (PPO) implementation, so that changes in placement, observation composition, and actuator count can be examined within the same learning framework. Local indicators guide node screening, and the sample-and-hold analysis connects bounded-input operation with the behavior of the deployed policies.

The experimental design separates observation size from coordinate identity, then examines every nonempty actuator subset. A targeted soft actor-critic (SAC) check tests whether the observation preference persists under another learning algorithm. Traditional feedback, linear quadratic regulator (LQR), SMC, and matched-budget SAC comparisons establish the broader performance context, with synchronization error and applied effort reported together. This sequence moves from explaining configuration differences to assessing their value in controller design.

Synchronized chaotic trajectories are additionally used in an image encryption and decryption demonstration based on Logistic-map masking and dynamic deoxyribonucleic acid (DNA) coding. This application examines the use of the generated trajectories beyond the control task; its statistical image results are evaluated separately from the synchronization evidence.

The main contributions are as follows:

- We identify an actuator-dependent observation preference by comparing all three paired four-dimensional subsets at Nodes 2 and 3. A targeted SAC check reproduces the direction of the original-versus-alternative subset error changes. An accuracy-constrained selection example then connects these observations to choices within a fixed candidate pool, checked on held-out initial conditions.

- We quantify the accuracy and effort costs of restricting actuation by evaluating every nonempty actuator subset. Comparisons with fixed-gain feedback, LQR, SMC, and matched-budget SAC distinguish configuration effects from controller-dependent performance, providing a common basis for interpreting the cost of single-input synchronization.

The state-boundedness result and post-training sensitivity checks support these comparisons at two distinct levels: bounded-input operation and local perturbation attenuation along deployed trajectories.

The remainder of this paper is organized as follows. Section 2 introduces the continuous-time Hopfield chaotic neural network and formulates the single-input pinning synchronization problem. Section 3 presents the local pinning-node selection analysis and the DRL-based control framework implemented by PPO. Section 4 reports the experimental setup, results, and discussion. Section 5 presents the application in image encryption. Finally, Section 6 concludes this paper and outlines possible future work.

2. Preliminaries

Hopfield's continuous neural network model proposed in 1984 [25] can be written as

$$C_i \frac{du_i}{dt} = \sum_{j=1}^n T_{ij} V_j - \frac{u_i}{R_i} + I_i, \quad (1)$$

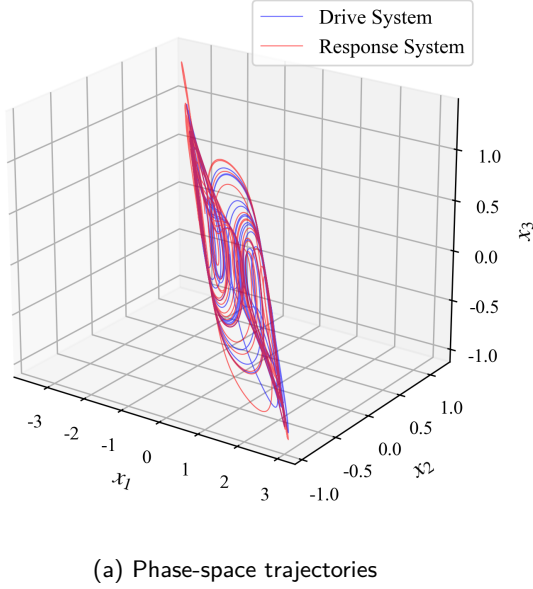
with the static nonlinear input–output mapping $V_i = g_i(u_i)$. Here, u_i denotes the internal state variable of the i th neuron, V_i is the neuron output, C_i and R_i represent the equivalent capacitance and resistance, respectively, T_{ij} denotes the synaptic connection weight, and I_i is the external bias input.

The symmetric coupling in the original Hopfield model supports an energy-based description of relaxation toward equilibrium. The chaotic network considered here instead uses asymmetric coupling, which removes that symmetry condition. Following the normalized Hopfield-type formulation in the control literature [26, 29], we specify the drive dynamics first and then introduce the controlled response and synchronization error.

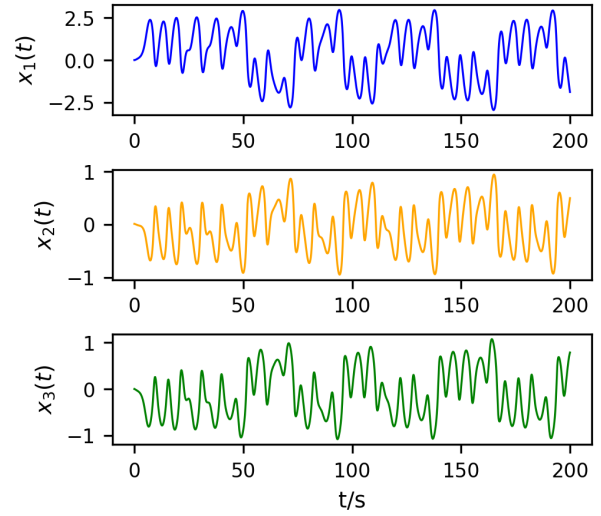
Introducing the parameter substitutions $a_i = \frac{1}{R_i C_i}$, $w_{ij} = \frac{T_{ij}}{C_i}$, and $b_i = \frac{I_i}{C_i}$, and assuming that all neurons share the same leakage coefficient ($a_i = 1$), neglecting the constant bias term ($\mathbf{b} = \mathbf{0}$), and choosing $\tanh(\cdot)$ to characterize the saturating nonlinearity of the neuron output, one obtains the continuous-time Hopfield-type neural network model used in the subsequent analysis:

$$\dot{\mathbf{x}}(t) = -\mathbf{x}(t) + W \tanh(\mathbf{x}(t)), \quad (2)$$

Here, a node denotes one scalar state coordinate of the Hopfield network, rather than an entire chaotic oscillator. The drive and response are two copies of this network. The state vector is $\mathbf{x}(t) = [x_1(t), x_2(t), x_3(t)]^T$. Following the



(a) Phase-space trajectories



(b) Time-domain trajectories

Figure 1: Uncontrolled phase-space trajectories of the drive and response Hopfield chaotic neural networks and time-domain state trajectories of the continuous-time Hopfield-type chaotic neural network.

three-neuron Hopfield-type chaotic model studied in [29], in this paper, the network weight matrix is specified as

$$W = \begin{bmatrix} -1.4 & 1.2 & -7.0 \\ 1.1 & 0.0 & 2.8 \\ 0.8 & -2.0 & 4.0 \end{bmatrix}. \quad (3)$$

The network in (2) is taken as the drive system. A single-input pinning controller is introduced into one selected response node, giving

$$\dot{\mathbf{y}}(t) = -\mathbf{y}(t) + W_r \tanh(\mathbf{y}(t)) + Bu(t), \quad (4)$$

where $\mathbf{y}(t) \in \mathbb{R}^3$ is the state of the response system; W_r is the weight matrix of the response system, which may differ from W to represent parameter uncertainty; $u(t) \in \mathbb{R}$ is the scalar control input; and $B \in \mathbb{R}^{3 \times 1}$ is the pinning-node selection vector, which determines the specific physical node into which the control signal is injected. Only one of its three components is equal to 1, while the others are 0.

For this parameter setting, the initial condition $\mathbf{x}(0) = (0, 0.01, 0)^T$, which has been used as a representative initial state in previous studies on Hopfield chaotic neural networks [29], is adopted to illustrate the basic chaotic behavior of the system. The fourth-order Runge–Kutta (RK4) trajectories illustrate the reported chaotic regime of this model. To further illustrate the uncontrolled drive–response behavior before applying the proposed pinning controller, two identical Hopfield chaotic neural networks are simulated with the same system parameters but different initial states sampled from Uniform($[-1, 1]^3$). As shown in Fig. 1(a), the two trajectories evolve around the same double-scroll chaotic attractor but do not naturally synchronize.

The time-domain trajectories of $x_1(t)$, $x_2(t)$, and $x_3(t)$ obtained from the representative initial condition $\mathbf{x}(0) =$

$(0, 0.01, 0)^T$ are shown in Fig. 1(b). The plotted irregular oscillations illustrate the drive dynamics in the reported chaotic regime.

Define the synchronization error as $\mathbf{e}(t) = \mathbf{y}(t) - \mathbf{x}(t)$. The ideal objective is vanishing synchronization error [2]. Subtracting the drive dynamics from the response dynamics gives

$$\dot{\mathbf{e}}(t) = -\mathbf{e}(t) + W_r \tanh(\mathbf{y}(t)) - W \tanh(\mathbf{x}(t)) + Bu(t). \quad (5)$$

The following definition mainly corresponds to the nominal synchronization control problem.

Definition 1. For the drive system (2) and the response system with pinning control (4), if under the action of the control law $u(t)$ one has $\lim_{t \rightarrow \infty} \|\mathbf{e}(t)\| = 0$, then the systems are said to achieve asymptotic synchronization.

This definition provides the ideal reference. The implemented objective is to reduce synchronization error using a fixed actuator location and bounded input. Here, practical synchronization denotes the observed low-residual behavior over the reported testing windows; it is assessed by full-horizon and terminal error metrics rather than an assumed asymptotic limit. Section 3.3 separately examines boundedness and post-training trajectory sensitivity. Full observation is the default, and reduced policy inputs are evaluated as a separate experimental factor.

3. PPO-Based Single-Input Pinning Control Method

The framework has three stages. Local dynamical and control indicators first identify candidate actuator locations.

For each tested location, PPO learns a bounded scalar feedback policy from the nominal simulator. Finally, the trained policy with its parameters held fixed is evaluated using the implemented sampled dynamics, with state boundedness and local trajectory sensitivity treated separately. The indicators inform the choice of channels to investigate; they are not inputs to the policy or terms in the training reward.

3.1. Local Pinning-Node Selection Analysis

Candidate nodes are compared using complementary local descriptions: transverse growth under ideal coordinate pinning, the LQR cost of a linear model evaluated at the origin, and an index based on the controllability matrix. These descriptions probe different aspects of actuator placement. They support qualitative screening before nonlinear evaluation, without assigning a combined numerical score.

For the local pinning-node analysis, we first consider the baseline identical-system setting, where the drive and response systems share the same weight matrix, i.e., $W_r = W$, and no external disturbance is imposed. When the synchronization error approaches zero, i.e., $\mathbf{e} \rightarrow \mathbf{0}$, the nonlinear term can be expanded to the first order around the synchronization manifold. This gives the following local linear approximation of the error system [34]:

$$\dot{\mathbf{e}}(t) \approx [-I + W D(\mathbf{x}(t))] \mathbf{e}(t) + B u(t), \quad (6)$$

where I is the 3×3 identity matrix, W is the weight matrix of the drive system, $u(t) \in \mathbb{R}$ is the scalar control input, and $B \in \mathbb{R}^{3 \times 1}$ is the control allocation vector indicating the node at which the control signal is injected. The diagonal matrix $D(\mathbf{x})$ is composed of the derivatives of the activation function:

$$D(\mathbf{x}) = \text{diag}(1 - \tanh^2(x_1), 1 - \tanh^2(x_2), 1 - \tanh^2(x_3)), \quad (7)$$

where x_1 , x_2 , and x_3 are the state variables of the drive system.

For single-input pinning control, different node selections correspond to different control allocation vectors:

$$B_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad B_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad B_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}. \quad (8)$$

Here, B_1 , B_2 , and B_3 represent the cases where the control input is injected into Nodes 1, 2, and 3, respectively.

First, the maximum conditional Lyapunov exponent (MCLE) [35] is used under an ideal coordinate-pinning assumption. If the selected error coordinate is constrained to remain identically zero, let S_i contain the two columns of I_3 associated with the remaining coordinates. Their variational equation is $\dot{\xi} = S_i^T [-I + W D(\mathbf{x}(t))] S_i \xi$. This idealized reduced system differs from finite-amplitude, full-error feedback through B_i . Let $\Psi_i(t, 0)$ be the fundamental matrix of this reduced variational system, with $\Psi_i(0, 0) = I_2$. Its largest growth rate is expressed as

$$\lambda_{\max}^{(c)}(B_i) = \limsup_{t \rightarrow \infty} \frac{1}{t} \ln \|\Psi_i(t, 0)\|_2, \quad i = 1, 2, 3, \quad (9)$$

where $\|\cdot\|_2$ is the induced Euclidean matrix norm, so the expression accounts for the most amplified perturbation direction. If $\lambda_{\max}^{(c)}(B_i) < 0$, small perturbations in the corresponding transverse subspace tend to decay locally; if $\lambda_{\max}^{(c)}(B_i) > 0$, locally divergent directions still exist in that subspace. In this work, the MCLE is used as a local reference index for node pre-screening under the ideal pinning assumption.

Second, to compare the local control difficulty associated with different pinning nodes, a time-invariant linear approximation is considered. Specifically, the Jacobian matrix is evaluated at the reference point $\mathbf{x} = \mathbf{0}$, where $D(\mathbf{0}) = I$, leading to $A = W - I$. Based on this local approximation, the LQR framework [36] is used here as an auxiliary indicator for comparing candidate pinning nodes. An LQR controller derived from the same local model is separately included as a model-based baseline in Section 4.

For each candidate input vector B_i , the standard LQR performance index is introduced as

$$J = \int_0^\infty (\mathbf{e}^T Q \mathbf{e} + u^T R u) dt, \quad (10)$$

where Q is the state weighting matrix and R is the input weighting matrix. The corresponding local state-feedback control law is

$$u = -K_i \mathbf{e}, \quad (11)$$

where the feedback gain matrix is given by

$$K_i = R^{-1} B_i^T P_i, \quad (12)$$

and P_i is determined by the continuous algebraic Riccati equation associated with (A, B_i) .

In this paper, the Riccati-based cost index is defined as

$$J_R(B_i) = \text{tr}(P_i), \quad (13)$$

which sums the costs associated with the coordinate directions of the local linear model. For a zero-mean initial error with identity covariance, the expected optimal linear cost is $\mathbb{E}[\mathbf{e}(0)^T P_i \mathbf{e}(0)] = \text{tr}(P_i)$. Thus the index compares nodes under a common state scaling and weighting, while $\|K_i\|$ describes local feedback strength. Their role is to characterize the local linear model; the applied effort of the saturated controllers is measured separately in the experiments.

Finally, to measure how effectively the control action propagates from different nodes to the entire local state space, the same local linearization matrix $A = W - I$ is used. Based on this matrix, the controllability matrices corresponding to the three candidate pinning nodes are constructed as [37]

$$C_i = [B_i, AB_i, A^2 B_i], \quad i = 1, 2, 3, \quad (14)$$

and the local control volume index is defined as

$$V_i = \det(C_i)^2. \quad (15)$$

Here, C_i characterizes the ability of the control input injected through the i th node to propagate over the entire local state space. Since this determinant-based index depends on the chosen state scaling, it is used only as a relative indicator of control influence among candidate nodes under the same coordinate system.

Table 1

Local auxiliary indicators for candidate screening.

Pinning Node	$\lambda_{\max}^{(c)}$	J_R	$\ K_i\ $	V_i
Node 1	+0.5858	280.2686	20.3937	0.479
Node 2	+1.5892	15.5170	6.7286	262.18
Node 3	-0.5890	7.8327	4.3829	290.77

For the MCLE calculation, the drive initial state is

$$\mathbf{x}(0) = (0.5, -0.01, 0.2)^T.$$

The drive is integrated for 1000 time units, with the first 50 discarded. The state and reduced variational equations use RK4 with renormalization by factorization into an orthogonal matrix Q and an upper triangular matrix R (QR factorization) at each step. Step sizes 0.01, 0.005, and 0.0025 give the following ranges:

$$\begin{aligned} \text{Node 1: } & [0.5858, 0.5982], \\ \text{Node 2: } & [1.5854, 1.5900], \\ \text{Node 3: } & [-0.5890, -0.5841]. \end{aligned}$$

The signs are unchanged across the three step sizes. The table reports the estimates obtained with the finest step size; they remain finite-time numerical estimates.

Table 1 shows that Node 3 has a negative ideal-pinning MCLE estimate, the smallest J_R and $\|K_i\|$, and the largest V_i . These consistent local indicators motivate its selection as a preferred candidate in this system, subject to nonlinear closed-loop evaluation.

Node 2 has a larger controllability determinant index and a lower Riccati cost than Node 1, although its ideal-pinning transverse growth estimate is higher. The indices therefore do not yield an identical ordering of these two channels. Node 1 has the largest Riccati cost and gain norm and the smallest determinant index; these properties motivate testing it as a less favorable candidate under the common bounded-input learning setup.

The MCLE alone does not order Nodes 1 and 2 in the same way as the other indicators. Node 2 is retained ahead of Node 1 because its Riccati cost and gain norm are substantially smaller and its local volume is larger. This combines the complementary local indicators into a qualitative screening preference. Using $>$ to denote screening preference, the resulting qualitative order for this system is

$$\text{Node 3} > \text{Node 2} > \text{Node 1}. \quad (16)$$

The resulting experimental design retains all three single-node policies for nominal comparison. Node 3 supplies the

favorable reference channel, and Node 2 supplies a contrasting channel for the subsequent observation and robustness assessments.

All three controllability matrices evaluated at the origin have rank three. Accordingly, a positive ideal-pinning MCLE does not imply uncontrollability of (A, B_i) or rule out synchronization by a designed traditional feedback law. The designation “challenging” describes relative behavior in this model and the tested constrained controllers, not impossibility of linear control at Node 2.

The ideal-pinning exponent follows the drive trajectory, whereas the Riccati and determinant indices evaluate the Jacobian at the origin. Their agreement for Node 3 is useful screening evidence, but a broader relation between these local indicators and learned performance would require additional networks and parameter settings. Section 4 checks their relevance to the present nonlinear system.

3.2. PPO-Based Control Formulation and Training Procedure

Once the actuator location is fixed, the remaining design task is to map the available measurements to a bounded control input. With fixed nominal parameters, the sampled drive–response state defines a continuous-state, continuous-action Markov decision process. The full observation $(\mathbf{x}, \mathbf{e})$ determines both system states because $\mathbf{y} = \mathbf{x} + \mathbf{e}$. A policy is trained separately for each actuator location; it does not switch the controlled node during an episode. Reduced observations are treated separately in Section 4.2.

To enable the agent to perceive both the instantaneous dynamical state of the drive system and the current synchronization deviation, the full observation vector is defined as

$$O_t = \frac{1}{c_o} [x_1(t), x_2(t), x_3(t), e_1(t), e_2(t), e_3(t)]^T, \quad (17)$$

where $c_o = 5.0$ is the fixed observation scaling used in implementation. The notation c_o is used here to avoid confusion with the Gaussian noise intensity σ used in the robustness experiments.

For PPO, the sampled Gaussian policy action during training, or its deterministic mean during evaluation, is clipped to $[-1, 1]$ before deployment. Denoting this clipped action by a_t , it is mapped to the physical control input as

$$u(t) = \kappa a_t, \quad (18)$$

where κ is the control scaling factor, and $\kappa = 4$ is adopted in this paper.

Let $t_k = k\Delta t$ denote the control-update times. The applied input $u_k = \kappa a_k$ is held constant on $[t_k, t_{k+1})$. The environment then advances the drive and response and returns a reward based on the updated error:

$$r_k = -0.1 \|\mathbf{e}(t_{k+1})\|_1 = -0.1 \sum_{i=1}^3 |e_i(t_{k+1})|. \quad (19)$$

Maximizing the discounted reward promotes small synchronization errors throughout the rollout. The reward contains

no control-effort penalty; input magnitude is constrained by action clipping, and effort is measured separately during evaluation. The same error reward is retained in the reduced-observation and multi-input experiments to keep their learning objectives consistent.

PPO is selected for its direct Gaussian-policy parameterization and alternating rollout and minibatch-update implementation [38]. These features fit the sampled scalar-feedback task. The SAC comparison in Table 13 assesses the sensitivity of the results to the learning algorithm.

To learn a control policy in the above continuous-action setting, this paper employs the PPO algorithm [38]. PPO is an Actor–Critic method whose clipped surrogate moderates policy optimization through the action-probability ratio. This likelihood-ratio clipping is distinct from actuator clipping. The PPO objective function is written as

$$L(\theta) = \hat{\mathbb{E}}_t [\min(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)], \quad (20)$$

where

$$\rho_t(\theta) = \frac{\pi_\theta(\tilde{a}_t|O_t)}{\pi_{\theta_{\text{old}}}(\tilde{a}_t|O_t)}. \quad (21)$$

Here $\tilde{a}_t$ is the unclipped Gaussian sample stored for the policy update, whereas $a_t = \text{clip}(\tilde{a}_t, -1, 1)$ is applied to the environment. The ratio $\rho_t(\theta)$ compares the new and old action likelihoods, ϵ is the PPO clipping hyperparameter, and $\hat{A}_t$ is obtained using generalized advantage estimation (GAE). Optimization aims to reduce the discounted synchronization-error cost; attainment of its global minimum is not asserted.

As shown in Fig. 2, PPO follows an interaction–update training process. The agent first interacts with the environment using the current policy and stores the collected states, actions, rewards, and value estimates in a trajectory buffer. The advantage function is then estimated, typically using GAE, and the policy network is updated by maximizing a clipped surrogate objective. Meanwhile, the value network is trained by minimizing the mean squared error between the predicted and target state values. The clipped surrogate moderates likelihood-ratio changes during optimization; the deployed closed-loop behavior is assessed separately below.

The overall training procedure is summarized in Algorithm 1. In each iteration, trajectories are first collected by executing the current policy in the error-system environment. The collected data are then used to estimate the advantage function and update the policy network with the PPO clipped objective. The value network is updated by minimizing the squared value loss. After repeated interaction and optimization, the trained policy is used as the single-input pinning controller for the response system.

3.3. Boundedness and Post-Training Local Numerical Assessment

The trained policy is now treated as a fixed feedback law. Its assessment separates two questions: whether the controlled states remain bounded, and how nearby response

trajectories evolve under the deployed controller. The first follows analytically from the dissipative Hopfield dynamics and bounded actuation. The second is examined numerically through the closed-loop control-step Jacobian, using the same sample-and-hold implementation as in deployment.

Because the normalized action satisfies $a_t \in [-1, 1]$ and the physical action is defined by (18), the deployed control input satisfies

$$|u(t)| \leq \kappa, \quad \kappa = 4. \quad (22)$$

Moreover, each single-node pinning vector has $\|B_i\|_2 = 1$.

Proposition 3.1 (Boundedness of the controlled drive–response trajectories). *Consider the drive system (2) and response system (4), where W and W_r are finite constant matrices and the control input is measurable and satisfies (22). This includes the deployed piecewise-constant sample-and-hold input. For every finite initial condition, the corresponding trajectories are forward complete and uniformly ultimately bounded. In particular,*

$$\limsup_{t \rightarrow \infty} \|\mathbf{x}(t)\|_2 \leq \sqrt{3}\|W\|_2, \quad (23)$$

$$\limsup_{t \rightarrow \infty} \|\mathbf{y}(t)\|_2 \leq \sqrt{3}\|W_r\|_2 + \kappa. \quad (24)$$

Proof. Since $\|\tanh(\mathbf{z})\|_2 \leq \sqrt{3}$ for every $\mathbf{z} \in \mathbb{R}^3$, the Cauchy–Schwarz inequality and the induced matrix norm give

$$\begin{aligned} D^+ \|\mathbf{x}\|_2 &\leq -\|\mathbf{x}\|_2 + \sqrt{3}\|W\|_2, \\ D^+ \|\mathbf{y}\|_2 &\leq -\|\mathbf{y}\|_2 + \sqrt{3}\|W_r\|_2 + \|B_i\|_2 |u(t)| \\ &\leq -\|\mathbf{y}\|_2 + \sqrt{3}\|W_r\|_2 + \kappa, \end{aligned} \quad (25)$$

where D^+ is the upper right Dini derivative, $\|B_i\|_2 = 1$, and (22) has been used. Applying the scalar comparison lemma to (25) yields

$$\|\mathbf{x}(t)\|_2 \leq e^{-t}\|\mathbf{x}(0)\|_2 + \sqrt{3}\|W\|_2(1 - e^{-t}). \quad (26)$$

$$\|\mathbf{y}(t)\|_2 \leq e^{-t}\|\mathbf{y}(0)\|_2 + \left(\sqrt{3}\|W_r\|_2 + \kappa\right)(1 - e^{-t}). \quad (27)$$

Taking the upper limit as $t \rightarrow \infty$ gives (23) and (24). For a prescribed bounded measurable input, the vector field is globally Lipschitz in the state and has bounded forcing. The explicit bounds preclude finite escape; the sampled feedback can consequently be extended over successive control intervals for all future times. $\square$

For the nominal matched case $W_r = W$, the matrix in (3) has $\|W\|_2 = 8.9524$. Proposition 3.1 therefore gives the conservative bounds $\limsup_{t \rightarrow \infty} \|\mathbf{x}(t)\|_2 \leq 15.5059$ and $\limsup_{t \rightarrow \infty} \|\mathbf{y}(t)\|_2 \leq 19.5059$. These worst-case norm estimates establish non-divergence and provide conservative envelopes for the trajectories. In particular, boundedness of

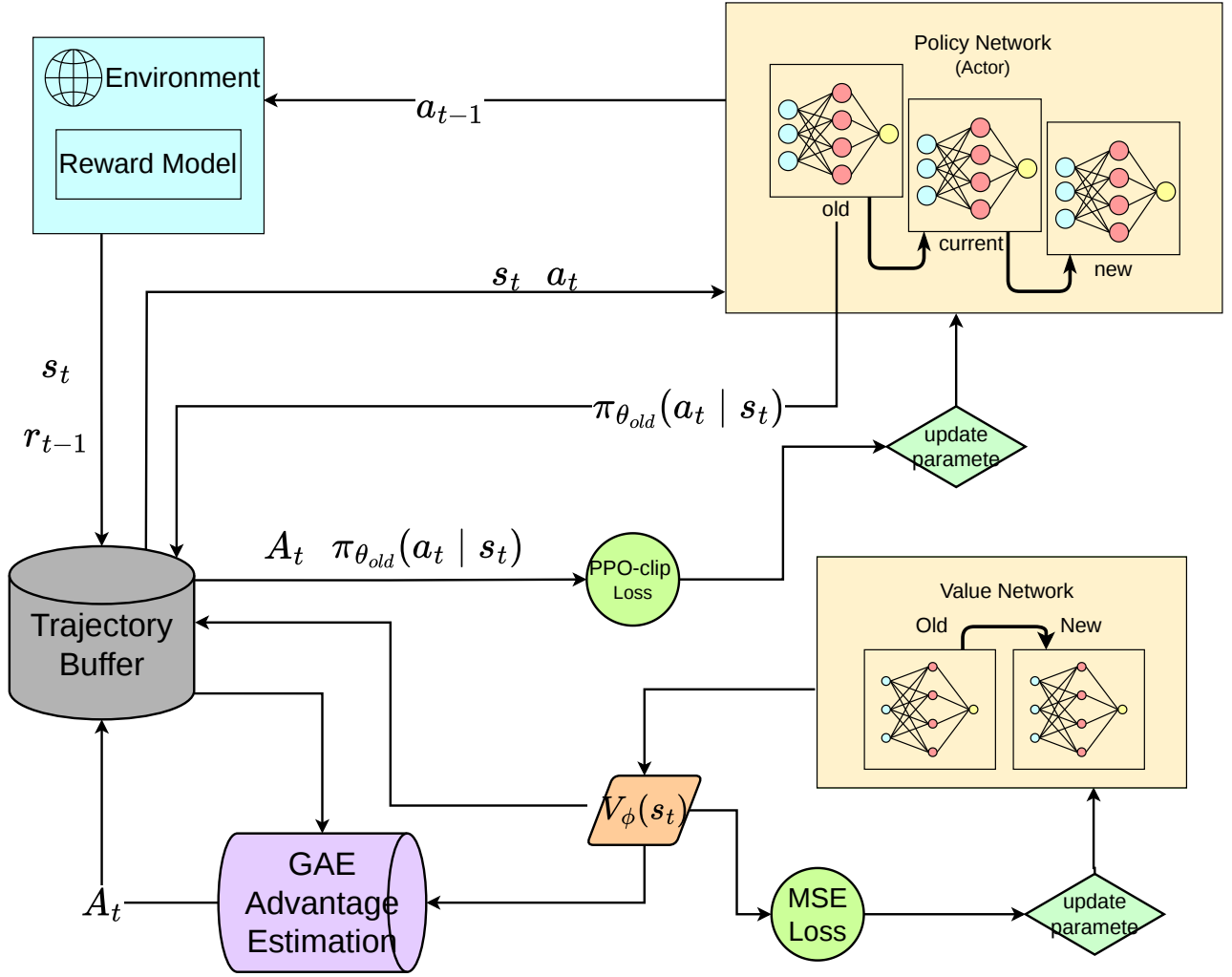


Figure 2: General training process of the PPO-based DRL framework.

both trajectories alone does not imply that the synchronization error converges to zero.

The MCLE $\lambda_{\max}^{(c)}(B_i)$ in Section 3.1 and the closed-loop exponent introduced below have different purposes. The former is a pre-training node-screening indicator derived from a locally linearized transverse model under the ideal pinning assumption and does not include the learned PPO policy. By contrast, the latter is evaluated after training from the actual deployed sample-and-hold closed-loop map. Consequently, their numerical values should not be directly compared or interpreted as equivalent stability measures.

The implementation uses a sample-and-hold controller: one action is evaluated every five RK4 substeps and is held constant over the control interval $\Delta t = 0.05$. Accordingly, local behavior of the trained controller is assessed using the actual discrete control-step map

$$\mathbf{x}_{k+1} = \Phi_x(\mathbf{x}_k), \quad \mathbf{e}_{k+1} = G_i(\mathbf{x}_k, \mathbf{e}_k; \pi_{\theta,i}), \quad (28)$$

where G_i includes the deterministic mean action of the trained policy, action clipping, the physical scale κ , and the five RK4 substeps. The transverse Jacobian of the deployed

map is

$$M_{i,k} = \frac{\partial G_i}{\partial \mathbf{e}}(\mathbf{x}_k, \mathbf{e}_k; \pi_{\theta,i}). \quad (29)$$

The asymptotic largest conditional Lyapunov exponent is defined by

$$\lambda_i^{\text{cl}} = \limsup_{N \rightarrow \infty} \frac{1}{N \Delta t} \ln \|M_{i,N-1} M_{i,N-2} \cdots M_{i,0}\|_2. \quad (30)$$

Automatic differentiation and QR renormalization are used to obtain a finite-time estimate $\hat{\lambda}_i^{\text{cl}}$ of the largest conditional Lyapunov exponent over the evaluation interval. A negative estimate indicates that small linearized perturbations to the response state decay on average along the tested trajectory, with the drive trajectory held fixed. This local decay does not imply zero synchronization error; therefore, the exponent and the synchronization residual are reported separately.

The numerical assessment was conducted for Nodes 2 and 3 using three independently trained policies for each node. Each policy was evaluated from ten initial conditions,

giving 30 closed-loop trajectories per node and 60 trajectories in total. Each trajectory contained 2000 control intervals, and the first 100 intervals were discarded as a transient, leaving 95 time units for exponent accumulation. Table 2 summarizes the largest conditional Lyapunov exponent and the post-transient synchronization residual. The reported mean error is obtained by first averaging $\|\mathbf{e}_k\|_2$ over the post-transient portion of each trajectory and then averaging over the 30 trajectories for the corresponding node.

No state guard was activated along these trajectories, so omitting that inactive operation from the differentiated map does not change the local derivative there. Action clipping is nonsmooth at its thresholds; the numerical derivatives describe behavior away from those thresholds.

Table 2: Finite-time conditional Lyapunov estimates for the deployed sample-and-hold PPO controllers.

Node	$\hat{\lambda}_i^{\text{cl}}$	Range	Mean $\ \mathbf{e}\ _2$
2	-1.117	[-1.317, -0.872]	0.0276
3	-0.581	[-0.592, -0.557]	0.0075

The finite-time estimate is negative for all 60 evaluated trajectories. A shorter check using 600 control intervals produced the same signs and similar aggregate values, supporting numerical consistency over the examined windows. Node 2 has a more negative mean estimate, whereas Node 3 has a smaller post-transient error. The two measurements describe complementary properties: local perturbation decay and residual synchronization error. In particular, the policy architecture does not impose $\pi_{\theta,i}(\mathbf{x}, \mathbf{0}) = 0$. Thus, the synchronization manifold need not be invariant. Here “practical synchronization” describes the observed low-residual behavior over the evaluation window.

As a complementary multi-step check, a positive-definite metric P was fitted once per policy using a separate nominal rollout. In the induced norm $\|\mathbf{v}\|_P = (\mathbf{v}^T P \mathbf{v})^{1/2}$, the largest sampled gains of the ordered 40-step Jacobian products were 0.4882 and 0.5340 for Nodes 2 and 3, respectively; at 80 steps they were 0.0673 and 0.2802. These checks used 60 nominal trajectories of 2,000 control intervals, excluding the first 100 intervals. Forty control intervals correspond to two system time units. Direct paired-rollout checks with initial response perturbations up to 0.05 along signed coordinate directions also showed contraction at these horizons.

Together, the one-step and multi-step assessments explain how local perturbations can decay over finite windows even when individual steps amplify them. State boundedness supplies the analytical foundation, while trajectory sensitivity and measured residuals describe distinct aspects of the trained feedback behavior. The numerical assessment is restricted to the sampled trajectories rather than a certified neighborhood-wide error bound. Building on this distinction, the following experiments compare synchronization accuracy, transient recovery, and control effort across configurations and operating conditions.

Algorithm 1 PPO-based Single-Input Pinning Control for the Error System

Require: Error-system environment $\mathcal{E}$; policy network π_θ ; value network V_ϕ ; learning rate η ; clipping parameter ϵ ; discount factor γ ; GAE parameter λ .

Ensure: Optimized policy parameters θ .

- 1: Initialize policy parameters θ_0 and value-function parameters ϕ_0 .
- 2: Initialize the drive-response system and obtain the initial observation O_0 .
- 3: Define a trajectory buffer Γ to store observations, actions, rewards, and value estimates.
- 4: **for** $k = 0, 1, 2, \dots, K - 1$ **do**
- 5: Collect trajectories Γ_k by executing the current policy π_{θ_k} in the environment.
- 6: **for** each time step t in Γ_k **do**
- 7: Sample $\tilde{a}_t \sim \pi_{\theta_k}(\cdot|O_t)$; set $a_t = \text{clip}(\tilde{a}_t, -1, 1)$.
- 8: Store the unclipped sample for the policy update; apply $u(t) = \kappa a_t$.
- 9: Apply $u(t)$ to the response system and observe reward r_t and next observation O_{t+1} .
- 10: **end for**
- 11: Compute advantage estimates $\hat{A}_t$ using generalized advantage estimation.
- 12: Update policy parameters by maximizing the PPO clipped objective:

$$\theta_{k+1} = \arg \max_{\theta} \hat{\mathbb{E}}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

where

$$\rho_t(\theta) = \frac{\pi_\theta(\tilde{a}_t|O_t)}{\pi_{\theta_k}(\tilde{a}_t|O_t)}.$$

- 13: Update value-function parameters by minimizing the squared value loss:

$$\phi_{k+1} = \arg \min_{\phi} \hat{\mathbb{E}}_t \left[(V_\phi(O_t) - \hat{R}_t)^2 \right].$$

- 14: **end for**
 - 15: **return** trained policy π_θ .
-

4. Experimental Results and Analysis

The experiments address three questions: how actuator location affects PPO performance and observation demand; what changes when additional actuators become available; and how the trained PPO policies compare with alternative controllers under parameter mismatch [39], impulse disturbances, and process noise. The comparisons report synchronization error and applied control effort under the stated shared test conditions.

4.1. Experimental Setup and Evaluation Metrics

The programs were implemented in Python using Stable Baselines3. The workstation is equipped with an NVIDIA RTX 5070Ti graphics processing unit (GPU); the policy training scripts considered here use the central processing unit (CPU). PPO used a nominal training budget of 4×10^5 environment transitions, with four parallel environments and 512 steps per environment in each rollout.

During environment interaction, the continuous-time system dynamics were integrated using the RK4 method. The basic integration step size was set to 0.01 time units. To improve numerical stability, five RK4 substeps were performed within each control step. Therefore, each control update corresponded to an effective time interval of 0.05 time units. The maximum magnitude of the physical control input was constrained to 4 units.

The implementation also contains a numerical state guard at $[-10, 10]$ after RK4 substeps and disturbance updates. No guard activation was detected in the 2,940 reevaluated episodes covering the traditional baselines, matched-budget SAC comparison, and actuator-count ablation; the reported trajectories in these comparisons therefore evolve without state truncation.

At initialization, the initial states of both the drive system and the response system, denoted by $\mathbf{x}(0)$ and $\mathbf{y}(0)$, were sampled from the uniform distribution $[-1.0, 1.0]$. This common distribution provides varied initial synchronization errors and is held fixed across the compared methods. The reported results are conditional on this initialization range.

For the reinforcement learning configuration, both the policy network and the value network were represented by multilayer perceptrons (MLPs). The Adam optimizer was adopted with a learning rate of 3×10^{-4} , so as to balance exploration efficiency and policy-update stability.

Three traditional controllers are included to provide references of increasing model dependence. First, the original fixed-gain feedback baseline [5] is retained without retuning:

$$u(t) = \begin{cases} -4, & -ke_i(t) < -4, \\ -ke_i(t), & -4 \leq -ke_i(t) \leq 4, \\ 4, & -ke_i(t) > 4, \end{cases} \quad (31)$$

where $e_i(t)$ denotes the synchronization error at the controlled node i . This baseline represents a simple local-error controller rather than an optimally tuned linear design. The previously selected gain $k = 8.0$ is used throughout all reported experiments.

Second, a continuous-time LQR baseline is obtained from the error model linearized at the origin in Section 3. With $Q = I_3$ and $R = 1$, its saturated control law is

$$u_{\text{LQR},i}(t) = \text{sat}_4[-K_i \mathbf{e}(t)], \quad (32)$$

where $K_i = R^{-1} B_i^T P_i$ and P_i is the stabilizing solution of the continuous algebraic Riccati equation associated with

(A, B_i) . The resulting gains are

$$\begin{aligned} K_2 &= (-0.116727, 3.871191, -5.502161), \\ K_3 &= (-0.171698, -2.085641, 3.850986). \end{aligned}$$

Unlike fixed-gain feedback, LQR uses all error components and the nominal local model. The principal error-effort assessment includes the design with $R = 0.01$ selected using separate validation initial conditions in Table 10. The original $R = 1$ design is retained in the scenario curves as a reference for weight sensitivity, not as the strongest LQR comparator. The resulting feedback is evaluated on the nonlinear system with actuator saturation.

Third, a boundary-layer SMC is included as a nonlinear model-based baseline [7, 8]. Define

$$\mathbf{g}_0(\mathbf{x}, \mathbf{y}) = -\mathbf{e} + W [\tanh(\mathbf{y}) - \tanh(\mathbf{x})], \quad (33)$$

and select $c_i^T = K_i / (K_i B_i)$ so that $c_i^T B_i = 1$. The sliding variable and control law are

$$s_i = c_i^T \mathbf{e}, \quad u_{\text{SMC},i} = \text{sat}_4 \left[-c_i^T \mathbf{g}_0 - k_s \text{sat} \left(\frac{s_i}{\phi} \right) \right], \quad (34)$$

where $\text{sat}(z) = \max(-1, \min(1, z))$, $k_s = 16$, and $\phi = 0.5$. These parameters minimize the recorded validation score over $k_s \in \{4, 8, 12, 16\}$ and $\phi \in \{0.2, 0.5, 1, 2\}$, using ten validation initial conditions separate from the test seeds. Validation includes nominal, mismatch ($\eta = 1$), and impulse scenarios; the score averages their mean absolute errors normalized by 0.1, 0.15, and 0.45, respectively. The selected parameters are kept unchanged during testing. Thus, although its compensation term uses the nominal model, SMC parameter selection has access to non-ideal validation scenarios. The saturated, sampled implementation is evaluated empirically; the continuous-time ideal sliding condition is not asserted to hold when the actuator saturates.

All four methods use the same RK4 solver, control-update interval, initial-state distribution, testing horizon (400 updates, or 20 time units), and actuator bound $|u| \leq 4$. The PPO results aggregate three independently trained policies, each evaluated on 20 common test seeds; each deterministic traditional controller is evaluated on the same 20 test seeds. For each scenario, the initial conditions and exogenous random realizations are shared across controllers. In the existing environment, drawing a mismatch matrix consumes random numbers before initialization; hence a fixed seed does not imply identical initial states between $\eta = 0$ and $\eta > 0$. Comparisons between controllers are paired within each scenario.

PPO is trained only on the nominal system. LQR uses $\mathbf{e}$ and the nominal local model, while SMC additionally uses $\mathbf{x}$ and $\mathbf{y}$ for nonlinear compensation. PPO receives $(\mathbf{x}, \mathbf{e})/5$ without evaluating an analytical model during deployment; its training simulator nevertheless uses the nominal dynamics. Fixed-gain feedback uses only e_i . Consequently, these are comparisons under matched actuation constraints, not identical information or design objectives. In particular, the PPO reward penalizes synchronization error but not control

energy, whereas LQR explicitly penalizes both. Lower error at higher energy is therefore interpreted as a performance–effort tradeoff, not evidence of superior energy efficiency.

To evaluate the synchronization performance of different control methods, several metrics are introduced from the perspectives of overall synchronization accuracy, steady-state error, transient recovery capability, and control effort. Let $T = 400$ denote the number of control updates and $N = 3$ the system dimension. For the baseline and ablation comparisons, errors are recorded at the $T + 1$ sampling instants including initialization. The synchronization error vector is

$$\mathbf{e}(t) = \mathbf{y}(t) - \mathbf{x}(t) = [e_1(t), e_2(t), \dots, e_N(t)]^T. \quad (35)$$

The mean absolute error (MAE) is used to measure the average synchronization deviation over the whole testing interval:

$$\text{MAE} = \frac{1}{(T+1)N} \sum_{t=0}^T \sum_{i=1}^N |e_i(t)|. \quad (36)$$

MAE provides an intuitive measure of the overall error level by assigning equal weights to different time steps and state components.

The root mean square error (RMSE) is used to characterize the overall error level while being more sensitive to large deviations:

$$\text{RMSE} = \sqrt{\frac{1}{(T+1)N} \sum_{t=0}^T \sum_{i=1}^N e_i^2(t)}. \quad (37)$$

Compared with MAE, RMSE is more sensitive to transient large errors and is therefore useful for evaluating error fluctuations under parameter mismatch and impulse disturbance.

The final L1 error is used to measure the synchronization accuracy at the end of testing:

$$E_f = \|\mathbf{e}(T)\|_1 = \sum_{i=1}^N |e_i(T)|. \quad (38)$$

This metric reflects how closely the response system approaches the drive system at the final testing instant.

In the impulse disturbance experiment, the maximum post-pulse error is used to describe the largest error overshoot after the disturbance is injected:

$$M_p = \max_{t \geq t_p} \|\mathbf{e}(t)\|_1, \quad (39)$$

where t_p denotes the impulse injection step. The imposed state jump largely determines the immediate peak, so M_p alone is not interpreted as a controller-dependent recovery measure.

The post-pulse mean error is defined as the average global synchronization error from the impulse injection time to the end of testing:

$$E_{\text{post}} = \frac{1}{T - t_p + 1} \sum_{t=t_p}^T \|\mathbf{e}(t)\|_1, \quad (40)$$

where t_p is the impulse injection time and T is the final testing step.

The control energy is used to measure the cumulative control effort over the testing process, summing over all m active input channels:

$$E_u = \sum_{t=1}^T \sum_{j=1}^m u_j^2(t) \Delta t, \quad (41)$$

where $\Delta t = 0.05$ is the effective control update interval and $u_j(t)$ is the applied, saturated input during update t . The single-input comparisons have $m = 1$. The steady L1 error averages $\|\mathbf{e}(t)\|_1$ over samples $t = 320, \dots, 400$ (the final four time units), and differs from the terminal error E_f . Reported means first average the test initial conditions for each policy and then average policy seeds; the deterministic baselines have only the first level. Repeated initial conditions for the same trained policy are not treated as independent training replications.

In summary, MAE and RMSE are used to evaluate the overall synchronization performance, Final L1 Error is used to describe the terminal synchronization accuracy, M_p and E_{post} are used to characterize transient recovery capability under impulse disturbance, and E_u is used to assess the control cost.

4.2. Effects of Control Location, Observation Variables, and Input Count

We first fix the learning setup and vary the actuator location to test the screening preference from Section 3. The single control input is applied separately to Nodes 1, 2, and 3. The resulting comparison provides the starting point for asking how much feedback information each location needs within the same training budget.

Fig. 3 shows faster error decay and a smaller residual at Node 3 than at Node 2, while Node 1 gives the weakest learned performance under the common budget. These results motivate examining Nodes 2 and 3 more closely: both support low-residual synchronization with full observations, but their response to restricted feedback may differ.

To avoid relying only on a single error curve for qualitative judgment, MAE, RMSE, and Final L1 Error are further used to quantitatively compare the synchronization performance under different pinning nodes. The results in Table 3 further confirm the above observation. In terms of both average error and terminal error, Node 3 achieves the best overall synchronization performance. This result is generally consistent with the local auxiliary indicators in Table 1, including the $\lambda_{\text{max}}^{(c)}$, J_R , $\|K_i\|$, and V_i . Thus the favorable Node 3 screening result is reflected in the tested policies. Agreement within this network supports the use of the indicators as local diagnostics; it does not establish a general predictor of PPO performance.

The component-level surfaces in Fig. 4 complement the aggregate error metrics. They show decay toward low error at Nodes 3 and 2 and persistent fluctuations at Node 1 across the displayed initial conditions. Surface height represents

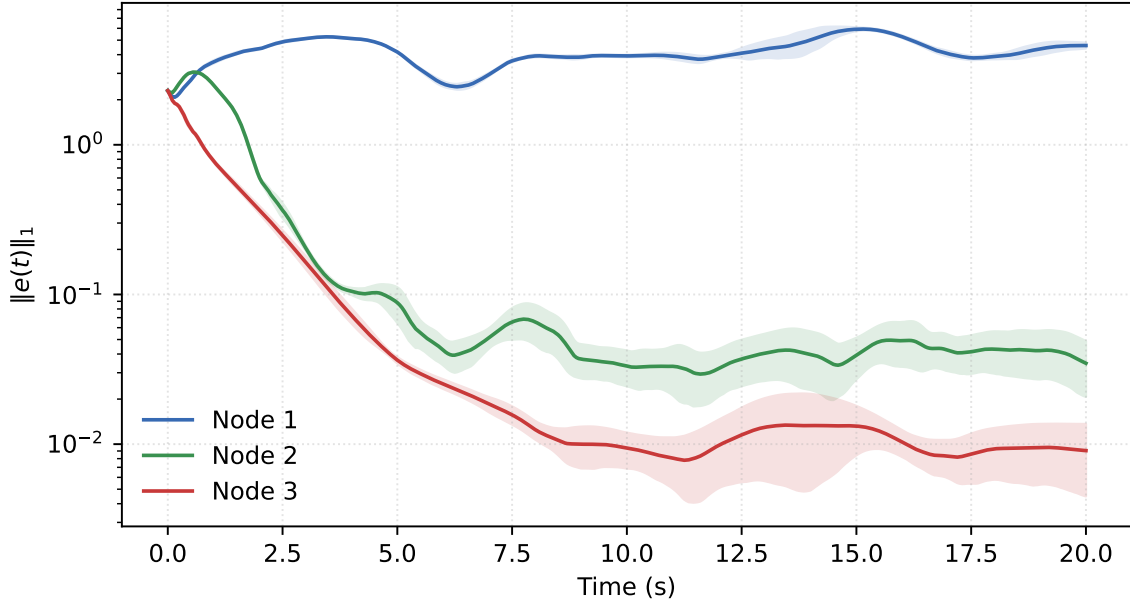


Figure 3: Synchronization error under different pinning nodes. Curves show training-run averages; shading indicates one sample standard deviation (SD) across runs.

Table 3

Synchronization performance under different pinning nodes. MAE and RMSE are computed from the three error components.

Node	MAE	RMSE	Final L1 Error E_f
Node 1	1.3937	1.6342	4.6052
Node 2	0.0930	0.2863	0.0347
Node 3	0.0431	0.1434	0.0091

signed component error, with time and test index on the other axes. Since test index is categorical, interpolation between trajectories serves only to visualize their differences; quantitative comparisons use Table 3.

Node 3 is retained as the favorable candidate and Node 2 as a contrasting, more challenging learned-control channel. Both are evaluated in the subsequent robustness and algorithm comparisons, allowing actuator placement to be considered alongside controller choice.

Partial observability is next considered to examine how the information demand of the learned policy depends on the selected pinning node [20]. Within each node, the actuation structure and training procedure are retained while a separate policy is trained for each observation configuration. Thus, this comparison concerns the performance attainable by the trained configurations, not the effect of removing measurements from one unchanged policy.

For Node 3, the full-observation policy has access to all drive states and synchronization errors. The four-dimensional and two-dimensional observations are

$$O_{4D} = \{x_1, x_3, e_1, e_3\}, \quad (42)$$

and

$$O_{2D} = \{x_3, e_3\}, \quad (43)$$

respectively. For Node 2, the corresponding observations are $\{x_1, x_2, e_1, e_2\}$ and $\{x_2, e_2\}$.

The four-dimensional observation contains the drive states and synchronization errors of Node 1 and the controlled node: $\{x_1, x_2, e_1, e_2\}$ for control at Node 2 and $\{x_1, x_3, e_1, e_3\}$ for control at Node 3. Observing Node 1 does not add a control input there; actuation remains restricted to the selected node. These subsets provide an intermediate comparison between full observation and observation of the controlled node alone. Only the policy inputs are reduced: the simulator still computes the training reward from all three error components in (19). The policy uses the current observation without a history of past measurements, so the reduced input need not provide a complete Markov state.

Fig. 5 and Table 4 show similar later-stage errors for the Node 3 full- and four-dimensional policies. The tested two-dimensional policies have higher MAE and terminal error. This difference reflects both the available measurements and the policies learned from them. To compare the increase in error within each node, normalize MAE by that node's six-dimensional value. The four- and two-dimensional penalties are approximately 10.5 and 43.7 at Node 2, versus 1.0 and 1.3 at Node 3. These descriptive ratios, calculated from the common-seed means in Table 4, quantify the actuator-dependent consequence of reducing the selected policy inputs.

With these particular subsets, Node 2 shows a larger degradation than Node 3. However, dimension reduction also changes coordinate composition, so these results alone cannot attribute the degradation to observation dimension.

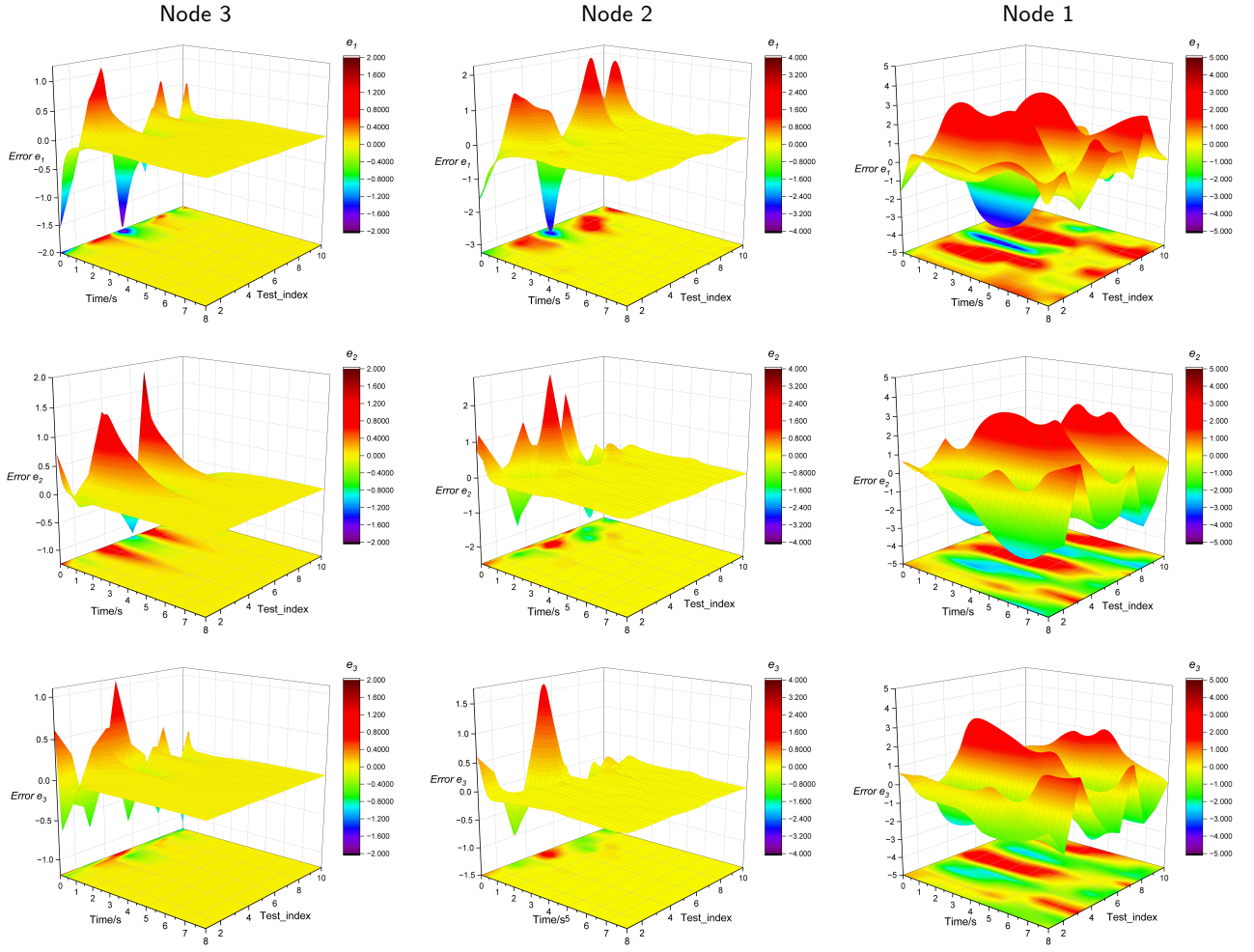


Figure 4: Three-dimensional synchronization-error surfaces for multiple initial conditions. Columns correspond to actuation at Nodes 3, 2, and 1; within each column, panels show e_1 , e_2 , and e_3 from top to bottom. Height and color represent signed error. Axis and color ranges differ between panels; quantitative comparisons across nodes should use the indicated scales.

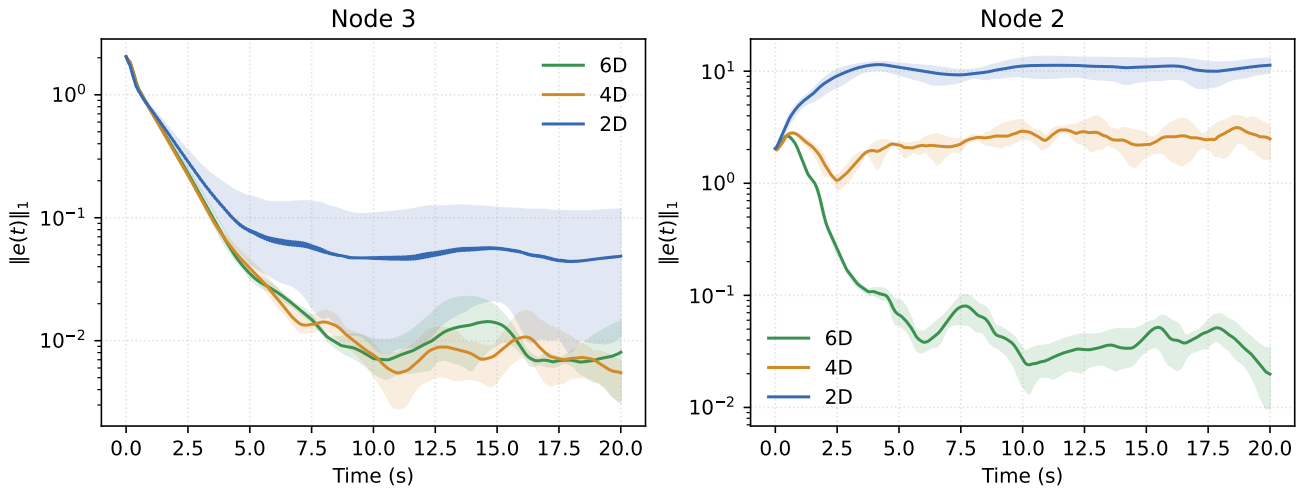


Figure 5: Comparison of L1 synchronization error under different observation configurations. Curves average three training-seed means, with ten common archived test initial conditions per seed; shading spans the minimum and maximum of the three seed means. This archived observation experiment uses fewer test initial conditions than the baseline and actuator-count comparisons.

Table 4

Synchronization performance under different observation settings, restricted to the same three training seeds and ten common test initial conditions. MAE is obtained by dividing the recorded time-averaged L1 error by $N = 3$. Observation components are specified in Eqs. 42–43.

Observation Setting	MAE	Final L1 Error
Node 3, 6D	0.0398	0.0081
Node 3, 4D	0.0390	0.0055
Node 3, 2D	0.0524	0.0487
Node 2, 6D	0.0761	0.0198
Node 2, 4D	0.7974	2.4874
Node 2, 2D	3.3271	11.3096

The following matched-dimension comparison examines this distinction.

To separate coordinate composition from input dimension, all three paired subsets $\mathcal{O}_{ij} = \{x_i, x_j, e_i, e_j\}$, with $ij \in \{12, 13, 23\}$, are evaluated at each of Nodes 2 and 3. The comparison covers two actuator locations and three coordinate pairs; each pair contains two drive states and their corresponding errors. Each configuration has three independently trained policies with a nominal budget of 4×10^5 transitions. All six configurations are evaluated on 20 new common initial conditions, with unchanged dynamics, reward, action bounds, and PPO settings. Every final checkpoint is retained. This evaluation set differs from that in the preceding dimension-reduction comparison.

Table 5

Complete paired four-dimensional observation comparison at Nodes 2 and 3. Entries are mean $\pm$ sample standard deviation across training-run means.

Node	Subset	MAE	E_u
2	$\mathcal{O}_{12}$	0.9104 ± 0.1312	9.60 ± 3.59
2	$\mathcal{O}_{13}$	0.6253 ± 0.0032	150.68 ± 38.64
2	$\mathcal{O}_{23}$	0.0866 ± 0.0059	9.64 ± 0.81
3	$\mathcal{O}_{12}$	0.5235 ± 0.0115	209.02 ± 25.40
3	$\mathcal{O}_{13}$	0.0438 ± 0.0009	6.85 ± 0.39
3	$\mathcal{O}_{23}$	0.0771 ± 0.0175	142.70 ± 154.59

Table 5 and Fig. 6 confirm the actuator-dependent effect of coordinate composition. Node 2 performs best among these paired subsets with $\mathcal{O}_{23}$, whereas Node 3 favors $\mathcal{O}_{13}$ in both mean MAE and energy. The subsets omitting the actuated node's state–error pair, namely Node 2/ $\mathcal{O}_{13}$ and Node 3/ $\mathcal{O}_{12}$, give much larger residuals and energy than these favorable configurations. However, including that pair is not sufficient for low residuals: Node 2/ $\mathcal{O}_{12}$ remains poor. Thus, the identity of the accompanying coordinate also matters. The results identify favorable observation combinations under the common finite-budget learning setup. At Node 3, $\mathcal{O}_{23}$ also exhibits substantial energy variability across training runs.

To check whether the direction of the observation-subset effect is specific to PPO, a targeted SAC comparison repeats the following pairs of four-dimensional observations: $\mathcal{O}_{12}$ versus $\mathcal{O}_{23}$ at Node 2, and $\mathcal{O}_{13}$ versus $\mathcal{O}_{23}$ at Node 3. Each configuration is trained independently in three runs at the same nominal budget of 4×10^5 transitions. The SAC settings used in the controller comparison below are unchanged apart from the observation input; no additional tuning or checkpoint selection is performed. All 12 final policies are evaluated deterministically on the same 20 initial conditions as the PPO paired-subset comparison. This check uses nominal dynamics only. The subsequent configuration-selection example uses PPO policies, so its candidate set is unchanged.

Table 6

SAC four-dimensional observation cross-check. Values are mean $\pm$ sample SD across training-run means, using the same test initial conditions as the PPO comparison in Table 5.

Node	Subset	MAE	E_u
2	$\mathcal{O}_{12}$	0.4637 ± 0.0659	40.73 ± 19.95
2	$\mathcal{O}_{23}$	0.1350 ± 0.0101	7.17 ± 0.69
3	$\mathcal{O}_{13}$	0.0562 ± 0.0067	7.54 ± 0.79
3	$\mathcal{O}_{23}$	0.0840 ± 0.0059	10.57 ± 0.54

Table 6 reproduces the direction of the PPO mean-error comparison. At Node 2, replacing $\mathcal{O}_{12}$ with $\mathcal{O}_{23}$ reduces SAC MAE from 0.4637 to 0.1350; at Node 3, replacing $\mathcal{O}_{13}$ with $\mathcal{O}_{23}$ increases it from 0.0562 to 0.0840. For each node, the same error ordering holds in all three SAC training runs. The agreement across PPO and SAC strengthens the evidence for placement-dependent observation preferences in the tested pairs. The SAC comparison focuses on these original and alternative subsets, complementing the complete paired-subset evaluation performed with PPO.

Control effort remains algorithm dependent. The Node 2 substitution reduces SAC energy from 40.73 to 7.17, whereas PPO energy changes little on average (9.60 to 9.64). At Node 3, energy increases for both algorithms, but the SAC increase (7.54 to 10.57) is much smaller than the PPO increase (6.85 to 142.70), which includes substantial training-run variability. Mean error ordering therefore transfers more consistently than the magnitude of the effort penalty.

To illustrate how measured performance can inform a resource-constrained choice, an offline screening example uses a prespecified eight-candidate pool, fixed before the paired-subset completion. A candidate c specifies the actuated node and its observation subset. The pool comprises Nodes 2 and 3, each with local two-dimensional observations, the original four-dimensional subset ($\mathcal{O}_{12}$ at Node 2 and $\mathcal{O}_{13}$ at Node 3), $\mathcal{O}_{23}$, and full six-dimensional observations. Each candidate retains all three training runs; configuration selection therefore accounts for variability across the retained runs. The allowed actuator set may be $\{2, 3\}$, $\{2\}$, or $\{3\}$. The subsequently evaluated Node 2/ $\mathcal{O}_{13}$ and Node 3/ $\mathcal{O}_{12}$ configurations are not included among these eight

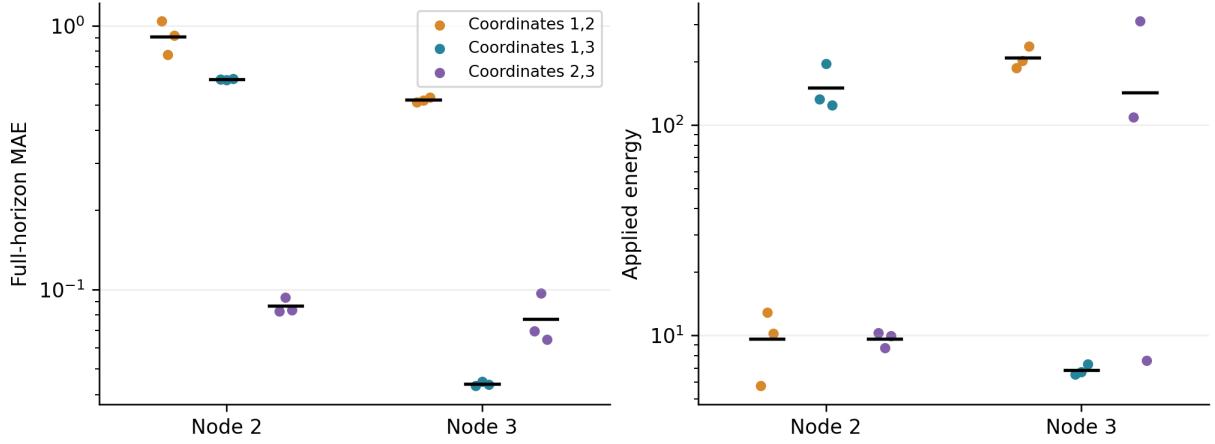


Figure 6: All paired four-dimensional subsets under single-node actuation. Each coordinate pair includes its drive states and error components. Points show individual training-run means; black marks indicate their average. Both error and energy are shown on logarithmic scales.

candidates; the complete paired-subset comparison and this limited selection example answer different questions.

For a trajectory, let z be the sample mean of $\|\mathbf{e}(t)\|_1$ over the final interval $16 \leq t \leq 20$. A trajectory passes when $z \leq \varepsilon$ and no state-protection boundary is reached. For training run r , its validation pass fraction is

$$\hat{p}_{c,r}(\varepsilon) = \frac{1}{n_v} \sum_{\ell=1}^{n_v} \mathbf{1}\{z_{c,r,\ell} \leq \varepsilon, g_{c,r,\ell} = 0\}, \quad (44)$$

where g counts state-protection boundary hits. A candidate is feasible only if $\min_r \hat{p}_{c,r} \geq 0.90$. Among feasible candidates with allowed actuation, the rule first minimizes observation dimension and then validation mean applied energy E_u . Remaining ties are resolved by a fixed candidate ordering. If none is feasible, no configuration is selected. Thus, fewer observed variables take priority over lower energy once the accuracy requirement is met.

Three error tolerances, $\varepsilon \in \{0.01, 0.05, 0.10\}$, are chosen with reference to the preceding experiments and fixed before configuration selection. Each tolerance specifies an acceptable mean synchronization error over the final four time units. For each tolerance, the rule uses 20 common validation initial conditions to identify configurations that meet the pass-rate requirement and select among them. The selected configurations are then evaluated on 40 separate test initial conditions drawn from the same initial-state distribution. Neither the tolerances nor the selections are adjusted using the test results. The system dynamics, nominal parameters, action bounds, and policy parameters remain unchanged throughout this procedure.

Table 7 shows which configurations are selected for each error tolerance. When either Node 2 or Node 3 can be controlled, the rule selects Node 3 with $\mathcal{O}_{13}$ at tolerances 0.05 and 0.10. When only Node 2 can be controlled, it selects $\mathcal{O}_{23}$ at 0.10. Every test trajectory of the selected configurations meets its corresponding error tolerance. At the stricter tolerance of 0.01, none of the eight candidates achieves the required validation pass rate.

Checking each training run separately avoids selecting a configuration whose average masks an unreliable run. For example, at tolerance 0.05, Node 2 with $\mathcal{O}_{23}$ has validation pass rates of 100%, 75%, and 100%. Although their average exceeds 90%, one run falls below the requirement, so this configuration is not selected at that tolerance. At tolerance 0.10, its selection over full observation reflects the priority given to fewer observed variables, rather than minimum energy: its validation mean energy is slightly higher than that of the six-dimensional configuration.

The procedure selects among the eight candidates according to nominal accuracy, observation dimension, and energy, then checks the choice on separate test initial conditions. Pass fractions describe the tested policies and sampled initial conditions; they do not establish the same pass rates for all possible initial conditions or newly trained policies. The tail-mean criterion measures average late-stage accuracy, whereas pointwise error limits and application-specific recovery requirements call for their own acceptance criteria. Shared initial conditions provide paired comparisons across policies.

Using the same validation data, changing the required per-run pass fraction from 90% to 80% or 100% leaves the selections unchanged. Giving energy priority over observation dimension changes only the Node 2 choice at $\varepsilon = 0.10$, from $\mathcal{O}_{23}$ to full observations. This illustrates why the resource priority must be stated explicitly; the candidate set and configurations chosen for independent testing remain unchanged.

We then examine whether the nominal choices retain their requested accuracy when response parameters change. The original selected Node 2/ $\mathcal{O}_{23}$ and Node 3/ $\mathcal{O}_{13}$ policies are evaluated without changing their parameters at their respective tolerances 0.10 and 0.05. For 40 new common initial conditions, response weights are set to $W_r = W \odot (1 + \eta U)$, where each entry of U is sampled uniformly from $[-1, 1]$ and $\eta \in \{0, 0.01, 0.03, 0.05\}$. The drive system is unchanged. Each initial condition uses the same mismatch

Table 7

Configurations selected using validation data and evaluated on separate test initial conditions. Test pass fractions are the minimum across three training runs, each evaluated on the same 40 test initial conditions. Error and energy average initial conditions and runs. A dash denotes no selection and hence no corresponding test; grouped tolerances have identical outcomes.

Allowed nodes	ϵ	Selected node / observations	Test pass	Test z	Test E_u
{2, 3}	0.01	—	—	—	—
{2, 3}	0.05, 0.10	3 / $\mathcal{O}_{13}$	100%	0.0086	5.03
{2}	0.01, 0.05	—	—	—	—
{2}	0.10	2 / $\mathcal{O}_{23}$	100%	0.0282	8.11
{3}	0.01	—	—	—	—
{3}	0.05, 0.10	3 / $\mathcal{O}_{13}$	100%	0.0086	5.03

Table 8

Performance of the selected configurations under response-weight mismatch, without retraining or reselection. Pass fractions are the minimum across three training runs, each evaluated on the same 40 new initial conditions. Mean tail error z and energy E_u average runs and initial conditions. Pass fractions track each configuration against its own previously fixed tolerance.

Configuration	ϵ	η	Minimum pass	Mean z	Mean E_u
2/ $\mathcal{O}_{23}$	0.10	0%	100.0%	0.0266	10.08
2/ $\mathcal{O}_{23}$	0.10	1%	97.5%	0.0402	10.11
2/ $\mathcal{O}_{23}$	0.10	3%	57.5%	0.0942	10.20
2/ $\mathcal{O}_{23}$	0.10	5%	35.0%	0.1571	10.39
3/ $\mathcal{O}_{13}$	0.05	0%	100.0%	0.0084	5.84
3/ $\mathcal{O}_{13}$	0.05	1%	97.5%	0.0181	5.85
3/ $\mathcal{O}_{13}$	0.05	3%	75.0%	0.0455	5.91
3/ $\mathcal{O}_{13}$	0.05	5%	42.5%	0.0743	6.01

direction across amplitudes and policies; an independent random stream preserves identical initial states. Zero weights remain zero. No retraining or configuration reselection is performed.

Table 8 shows that both configurations retain a minimum pass fraction of 97.5% at 1% relative mismatch, but fall below the original 90% requirement at 3% and 5%. No state-protection boundary is reached in these 960 trajectories. The drop in pass fraction identifies the sensitivity of the selected configurations to larger response-weight deviations and motivates incorporating the intended parameter range into configuration validation.

Having examined which feedback variables to retain under single-node actuation, we next assess the complementary resource choice: the number of actuators. To quantify the performance cost of restricting actuation to one node, nominal PPO synchronization is evaluated for every nonempty subset of the three nodes: {1}, {2}, {3}, {1, 2}, {1, 3}, {2, 3}, and {1, 2, 3}. For a subset S , the input matrix contains the corresponding columns of I_3 , and the policy produces $|S|$ actions. The six-dimensional observation, error-only reward, initial-state distribution, RK4 integration, policy hidden layers, PPO hyperparameters, and nominal training budget of 4×10^5 transitions are unchanged. Each configuration includes three independent training runs and is evaluated on 20 common test initial conditions. All seven configurations and all three final checkpoints per configuration are reported, including configurations with poor synchronization.

The same per-actuator bound $|u_j| \leq 4$ is imposed in every configuration, while E_u sums energy across all active channels. Thus, additional actuators provide both additional control directions and a larger admissible total input magnitude ($\|\mathbf{u}\|_2 \leq 4\sqrt{|S|}$). The comparison therefore measures the joint effect of additional control directions and increased aggregate control authority. Only nominal synchronization is tested in this ablation. Actuator count quantifies the structural actuation requirement.

Table 9 and Fig. 7 quantify the tradeoff. Single-node control at Node 3 gives MAE 0.0431 and energy 6.54, whereas {2, 3} gives 0.0205 and 8.15 and {1, 2, 3} gives 0.0118 and 10.91. Thus, one actuator retains low-residual synchronization with lower measured energy, but incurs a higher full-horizon error than these multi-input configurations. The steady L1 error does not decrease monotonically with actuator count: it is 0.0091 for {3}, compared with 0.0141 for {2, 3} and 0.0105 for {1, 2, 3}. Full-horizon accuracy and steady-state precision thus capture different consequences of increasing actuator availability.

The {1, 2} configuration has higher MAE and energy than {2} under the fixed training budget, while configurations containing Node 3 perform better in full-horizon MAE. This emphasizes the effect of actuator placement and finite-budget policy optimization. Although adding an actuator enlarges the admissible control set, exploiting that flexibility also depends on policy optimization within the available training budget. These results quantify the accuracy-effort

Table 9

Nominal PPO ablation over all nonempty actuator subsets. Values are mean $\pm$ SD across training-run means; energy is summed over all active inputs.

Controlled nodes	Inputs	MAE	Steady L1 error	E_u
{1}	1	1.3937 ± 0.0029	4.2825 ± 0.1571	266.49 ± 13.41
{2}	1	0.0930 ± 0.0022	0.0426 ± 0.0131	9.09 ± 1.53
{3}	1	0.0431 ± 0.0012	0.0091 ± 0.0033	6.54 ± 0.34
{1, 2}	2	0.1145 ± 0.0110	0.0679 ± 0.0212	20.61 ± 1.28
{1, 3}	2	0.0232 ± 0.0007	0.0119 ± 0.0019	10.93 ± 0.12
{2, 3}	2	0.0205 ± 0.0011	0.0141 ± 0.0040	8.15 ± 0.27
{1, 2, 3}	3	0.0118 ± 0.0007	0.0105 ± 0.0026	10.91 ± 0.17

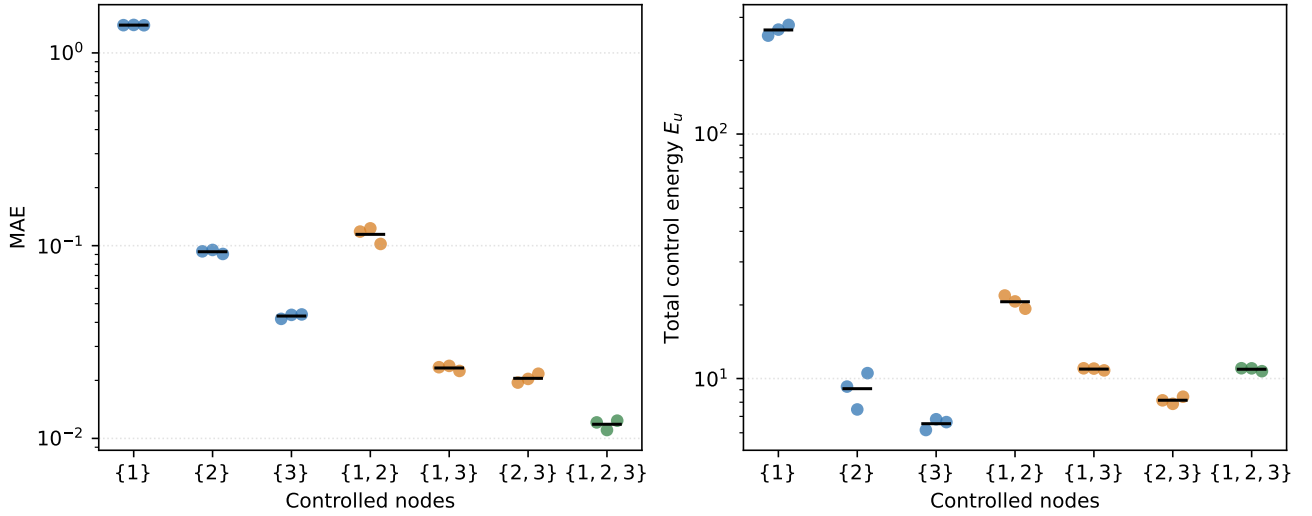


Figure 7: Nominal synchronization error and total control energy for all actuator subsets. Colored points show individual training-run means; black horizontal marks indicate their average. Blue, orange, and green indicate one, two, and three actuators, respectively.

tradeoff of single-input control as a resource-constrained design choice.

4.3. Controller Comparison and Robustness Assessment

The preceding experiments examine where to apply control and which variables to observe. We now assess how the resulting single-input PPO controllers compare with traditional feedback and SAC under nominal dynamics, parameter mismatch, impulses, and process noise. These comparisons test the performance and limitations of the learned feedback rather than seek a controller that is best in every setting. The scenario curves use the reference LQR weight $R = 1$; Table 10 additionally reports LQR with a weight selected using validation data.

For the LQR baseline, $Q = I_3$ is fixed and R is selected from $\{0.01, 0.1, 1, 10, 100\}$ using ten validation initial conditions and the normalized-MAE score specified for SMC. Both nodes select $R = 0.01$, which is kept fixed for the 20

separate test initial conditions. This selection includes nominal, mismatch, and impulse scenarios and does not penalize energy, whereas PPO training uses nominal dynamics only.

Table 10 compares PPO with the reference and selected LQR settings. At Node 2, LQR with $R = 0.01$ achieves lower full-horizon MAE in all four settings, but uses more energy under mismatch and impulse disturbance. Under nominal dynamics and noise, it has both lower MAE and lower energy than PPO.

At Node 3, the selected LQR has lower mean error and energy under nominal dynamics, mismatch, and noise. PPO has slightly lower post-impulse error but uses more energy. Thus, the learned controller does not uniformly outperform model-based feedback; its performance must be considered alongside the actuator and observation choices studied above.

We next examine how the controllers with unchanged parameters respond as model uncertainty increases [30]. The drive dynamics remain unchanged, while uncertainty in the

Table 10

Synchronization error and control energy of PPO and LQR. Both the reference weight $R = 1$ and the validation-selected weight $R = 0.01$ are reported. MAE covers the full simulation interval, including the impulse scenario.

Node	Scenario	PPO		LQR, $R = 1$		LQR, $R = 0.01$	
		MAE	E_u	MAE	E_u	MAE	E_u
2	Nominal	0.0930	9.09	0.0907	3.78	0.0772	10.58
2	$\eta = 2$	0.1703	11.85	0.2330	5.58	0.1486	14.60
2	Impulse	0.4339	17.51	0.5128	9.95	0.3368	31.23
2	$\sigma = 0.05$	0.2669	135.57	0.3742	11.67	0.2427	90.27
3	Nominal	0.0431	6.54	0.0510	2.65	0.0392	4.99
3	$\eta = 2$	0.0656	6.44	0.0989	3.56	0.0647	5.29
3	Impulse	0.2581	18.09	0.3330	7.49	0.2592	13.71
3	$\sigma = 0.05$	0.1312	69.96	0.1826	7.11	0.1264	23.27

response system is introduced through its weight matrix:

$$W_r = W + \eta(W \odot \Delta), \quad (45)$$

where η denotes the mismatch intensity coefficient, $\odot$ denotes the Hadamard product, and $\Delta = [\delta_{ij}]$ is a random perturbation matrix with

$$\delta_{ij} \sim U(-0.03, 0.03). \quad (46)$$

This formulation corresponds to a relative multiplicative mismatch. That is, each nonzero connection weight in the response system is perturbed around its nominal value, while zero entries remain unchanged. This modeling approach preserves the original network topology and is also consistent with practical parameter deviations caused by manufacturing errors, calibration bias, and component tolerances.

It should be noted that η is a normalized scaling coefficient rather than the direct percentage of mismatch. When $\eta = 1.0$, the relative perturbation range is approximately $\pm 3\%$ of the original weight; when $\eta = 2.0$, the range is approximately $\pm 6\%$. In this work, the following mismatch levels are considered:

$$\eta \in \{0.0, 0.5, 1.0, 1.5, 2.0\}. \quad (47)$$

Here, $\eta = 0.0$ corresponds to the nominal case without mismatch, while larger values represent stronger parameter uncertainty.

All controllers are evaluated without adaptation under the perturbed response matrices. PPO is trained without domain randomization; LQR is designed from the nominal linearization, and SMC uses nominal compensation with parameters selected on separate validation scenarios as described above.

At $\eta = 2$, PPO has lower Node 2 MAE than fixed-gain feedback, SMC, and the reference LQR, but not the validation-selected LQR or SAC. The selected LQR achieves its lower error with more energy than PPO, whereas SAC uses less energy. Node 3 generally shows smaller absolute

error differences. Figure 8 and Table 11 report the reference-controller results; Tables 10 and 13 provide the selected LQR and SAC comparisons.

In addition to parameter uncertainty, sudden impulse disturbance is also considered to evaluate the transient recovery capability of the controllers [31]. In the testing implementation, an impulse is added to all three drive-state components immediately after integration at step 200 ($t = 10$); the response state is not directly shifted. This tests recovery following an abrupt change of the synchronization reference. The impulse vector is set as

$$\mathbf{p} = [5, 5, 5]^T. \quad (48)$$

The same drive-state jump is applied for both actuator locations and all controllers. It tests recovery after a shared reference change; the resulting response can still depend on the chosen jump direction.

Because the imposed jump largely determines the immediate peak M_p , recovery is assessed using E_{post} together with energy. At Node 2, PPO recovers with lower mean error than fixed-gain feedback and SMC. The selected LQR and SAC achieve lower recovery error than PPO but use more energy; SAC also shows greater variation across training runs. At Node 3, the mean recovery errors of fixed-gain feedback, PPO, and the selected LQR are close, with lower energy for LQR. Figure 9 and Table 11 show the reference settings, including LQR with $R = 1$.

To further evaluate the robustness of the learned controller under persistent random perturbations, zero-mean Gaussian noise is added to the response system during testing, while the training stage remains noise-free. This setting tests the trained policy with its parameters held fixed under stochastic state perturbations absent from training. Specifically, after each control step and numerical integration, additive Gaussian noise is imposed on the response state:

$$\mathbf{y}_{k+1}^{\text{noisy}} = \mathbf{y}_{k+1} + \xi_k, \quad (49)$$

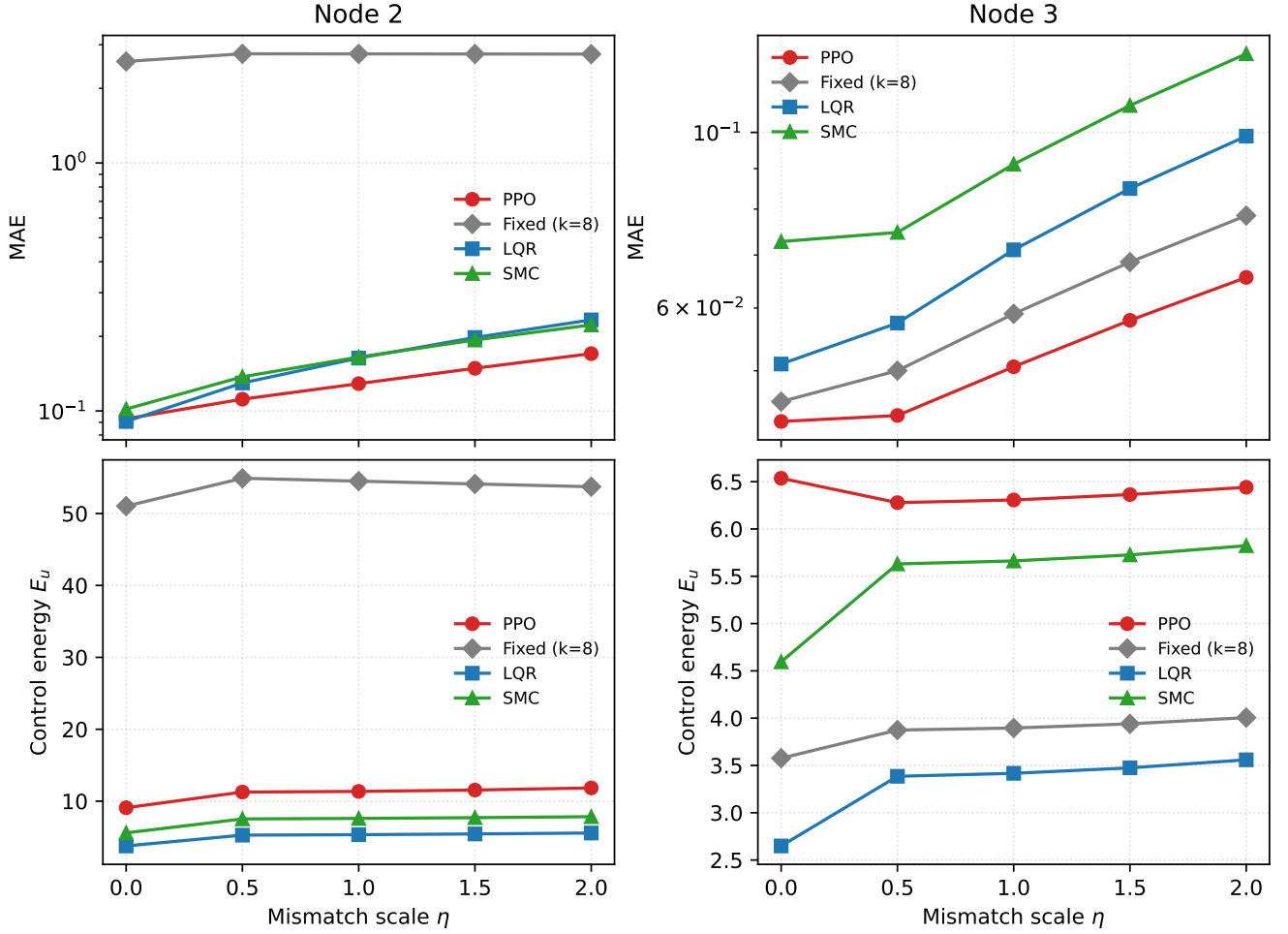


Figure 8: MAE and control energy of PPO, fixed-gain feedback ($k = 8$), LQR, and SMC under parameter mismatch. Curves show mean synchronization error and control energy.

Table 11

Representative mismatch and impulse-disturbance results.

Node	Method	Nominal		$\eta = 2$	Impulse	
		MAE	E_u	MAE	E_{post}	E_u
2	PPO	0.0930	9.09	0.1703	2.0800	17.51
	Fixed ($k = 8$)	2.5673	51.03	2.7481	8.6188	74.02
	LQR	0.0907	3.78	0.2330	2.5264	9.95
	SMC	0.1018	5.59	0.2223	2.7691	14.13
3	PPO	0.0431	6.54	0.0656	1.2969	18.09
	Fixed ($k = 8$)	0.0457	3.58	0.0785	1.2518	16.21
	LQR	0.0510	2.65	0.0989	1.6883	7.49
	SMC	0.0728	4.60	0.1257	1.7352	11.37

where

$$\xi_k \sim \mathcal{N}(\mathbf{0}, \sigma^2 I_3). \quad (50)$$

Here, σ denotes the noise intensity and I_3 is the 3×3 identity matrix. The tested noise levels are

$$\sigma \in \{0, 0.01, 0.02, 0.05\}. \quad (51)$$

At $\sigma = 0.05$, the selected LQR has lower Node 2 mean MAE and energy than PPO, while SMC has similar MAE and lower energy. PPO has lower MAE than SMC and the reference LQR. Figure 10 and Table 12 show how the reference controllers respond to increasing noise. Since noise is added to the evolving response state rather than

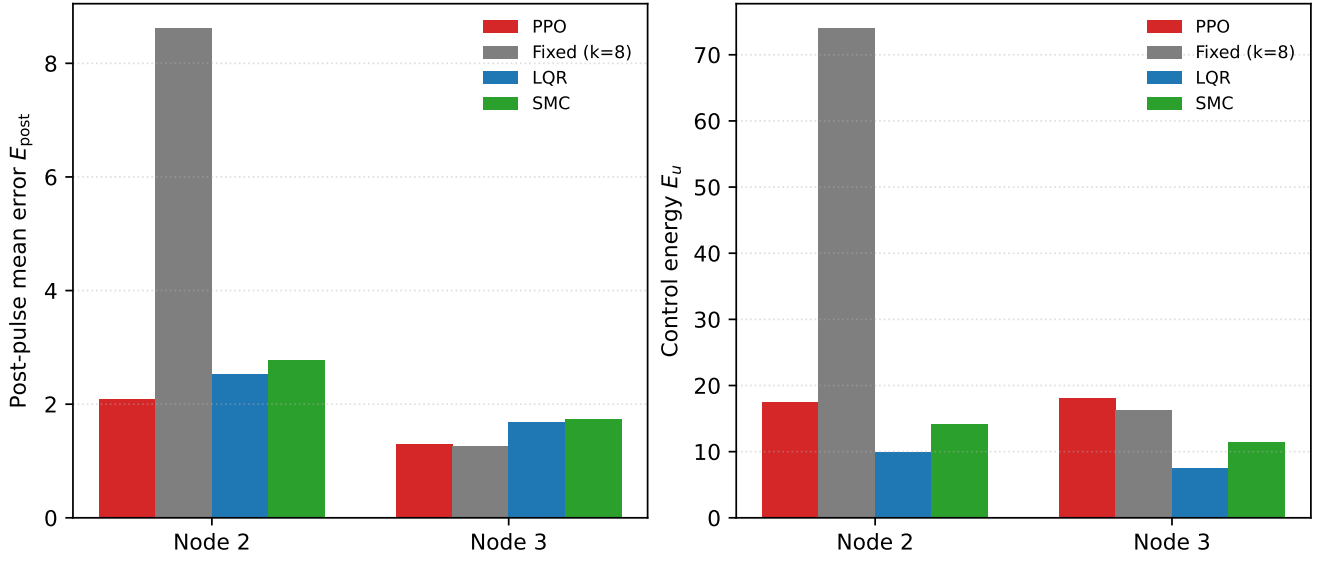


Figure 9: Post-pulse mean error and control energy for the reference settings under the same global impulse. LQR uses $R = 1$.

only to its measurements, these results describe sensitivity to process noise.

Table 12

Performance at the highest tested noise intensity, $\sigma = 0.05$.

Node	Method	MAE	Steady L1 error	E_u
2	PPO	0.2669	0.5383	135.57
	Fixed ($k = 8$)	2.6584	8.7280	56.99
	LQR	0.3742	0.8979	11.67
	SMC	0.3295	0.7298	140.49
3	PPO	0.1312	0.3183	69.96
	Fixed ($k = 8$)	0.1555	0.4000	11.09
	LQR	0.1826	0.4419	7.11
	SMC	0.2389	0.6029	92.51

To assess the dependence of the results on the learning algorithm, SAC [40] is evaluated on the same single-input Node 2 and Node 3 tasks. Both algorithms use the six-dimensional observation $(\mathbf{x}, \mathbf{e})/5$, reward $-0.1\|\mathbf{e}\|_1$, actuator bound $|u| \leq 4$, nominal training dynamics, and final checkpoints at a nominal budget of 4×10^5 environment transitions. Three independent training runs and 20 common test initial conditions per seed are used. All checkpoints are retained, and evaluation uses deterministic actions without online learning. The compared test settings are nominal dynamics, mismatch $\eta = 2$, the drive-state impulse at $t = 10$, and response process noise with $\sigma = 0.05$.

Both implementations use two hidden layers of width 64, a learning rate of 3×10^{-4} , batch size 128, and discount factor 0.99. PPO uses hyperbolic tangent hidden activations, four environments with 512 steps per rollout, ten optimization epochs, clip range 0.2, GAE parameter 0.95, and entropy coefficient 0.001. SAC uses rectified linear unit (ReLU) hidden activations, one environment, a replay buffer of 10^5

transitions, 5,000 initial exploration steps, one gradient update per subsequent transition, target-update coefficient $\tau = 0.005$, and automatic entropy adjustment with target entropy -1 . These settings are fixed across nodes and seeds. The budget matches environment interactions; gradient-update counts and training times follow each algorithm's update schedule. The comparison uses these fixed hyperparameter settings.

Table 13 and Fig. 11 show that neither learning algorithm gives the best result in every setting. At Node 2, PPO has lower nominal MAE, whereas SAC has lower MAE and energy under mismatch. After the impulse, SAC achieves lower mean recovery error but uses more energy and varies more across training runs. Under noise, their mean MAEs are close, with lower energy for SAC.

At Node 3, SAC has slightly lower mean MAE under nominal dynamics, mismatch, and noise, and lower energy in all four settings. PPO has lower post-impulse error. These differences qualify the performance of the chosen PPO implementation. More importantly for the observation-design question, the separate matched-subset SAC experiment reproduces the direction of the observation effects at both nodes.

The comparisons point to a practical distinction between actuator location and observation content. Node 2 benefits substantially from changing the four-dimensional subset, while Node 3 favors a different pair. The SAC check reproduces the direction of these error changes, strengthening the case for selecting feedback variables in relation to the actuated coordinate. The useful question is therefore not simply how many variables to retain, but which variables support the desired accuracy at the available actuator.

The configuration-selection example shows how to use this finding in controller design. The rule first excludes configurations that fail the accuracy requirement, then selects

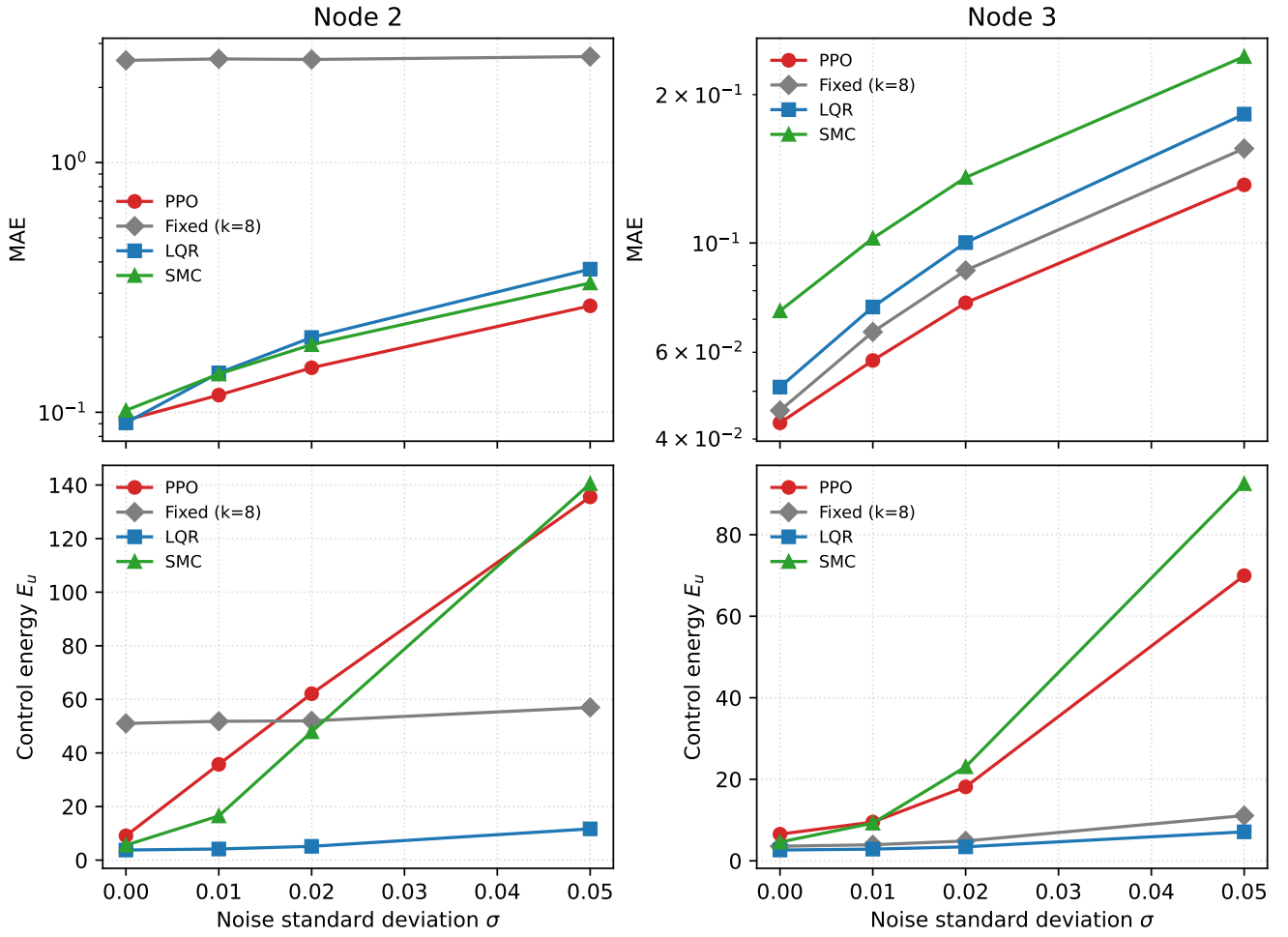


Figure 10: MAE and control energy of the four reference controllers under Gaussian-noise perturbation. LQR uses $R = 1$.

the smallest observation dimension and uses energy to distinguish equally sized configurations. Its mismatch tests show why a choice made under nominal conditions should be checked over the intended operating range. Adding actuators offers a different route to improved full-horizon accuracy, but also increases available control authority and measured total effort. Accuracy and applied effort can therefore be competing design requirements [41], even though the PPO reward used here optimizes error alone.

The controller comparisons establish the context for these design findings: PPO provides one learned implementation of bounded single-input feedback, not a uniformly superior alternative to LQR, SMC, or SAC. The main result concerns how actuator location and observation content interact, together with the accuracy and energy changes caused by adding control channels. Its scope is the stated three-state network, available measurements, and separately tested perturbations; other networks and combined disturbances require further evaluation.

Table 13

PPO and SAC at the same nominal budget of 4×10^5 environment transitions. Values are mean $\pm$ sample SD across training-run means. E_{post} is reported only for the impulse scenario.

Node	Scenario	Method	MAE	E_u	E_{post}
2	Nominal	PPO	0.0930 ± 0.0022	9.09 ± 1.53	—
2	Nominal	SAC	0.1173 ± 0.0046	6.27 ± 0.45	—
2	$\eta = 2$	PPO	0.1703 ± 0.0085	11.85 ± 1.60	—
2	$\eta = 2$	SAC	0.1513 ± 0.0037	9.31 ± 0.43	—
2	Impulse	PPO	0.4339 ± 0.0246	17.51 ± 1.19	2.0800 ± 0.1451
2	Impulse	SAC	0.4206 ± 0.0979	25.20 ± 9.79	1.9226 ± 0.5913
2	$\sigma = 0.05$	PPO	0.2669 ± 0.0049	135.57 ± 48.71	—
2	$\sigma = 0.05$	SAC	0.2632 ± 0.0052	86.13 ± 8.86	—
3	Nominal	PPO	0.0431 ± 0.0012	6.54 ± 0.34	—
3	Nominal	SAC	0.0416 ± 0.0010	4.60 ± 0.11	—
3	$\eta = 2$	PPO	0.0656 ± 0.0025	6.44 ± 0.30	—
3	$\eta = 2$	SAC	0.0644 ± 0.0009	5.07 ± 0.16	—
3	Impulse	PPO	0.2581 ± 0.0003	18.09 ± 3.50	1.2969 ± 0.0070
3	Impulse	SAC	0.2735 ± 0.0106	12.62 ± 0.53	1.3987 ± 0.0664
3	$\sigma = 0.05$	PPO	0.1312 ± 0.0066	69.96 ± 17.55	—
3	$\sigma = 0.05$	SAC	0.1273 ± 0.0013	50.40 ± 14.21	—

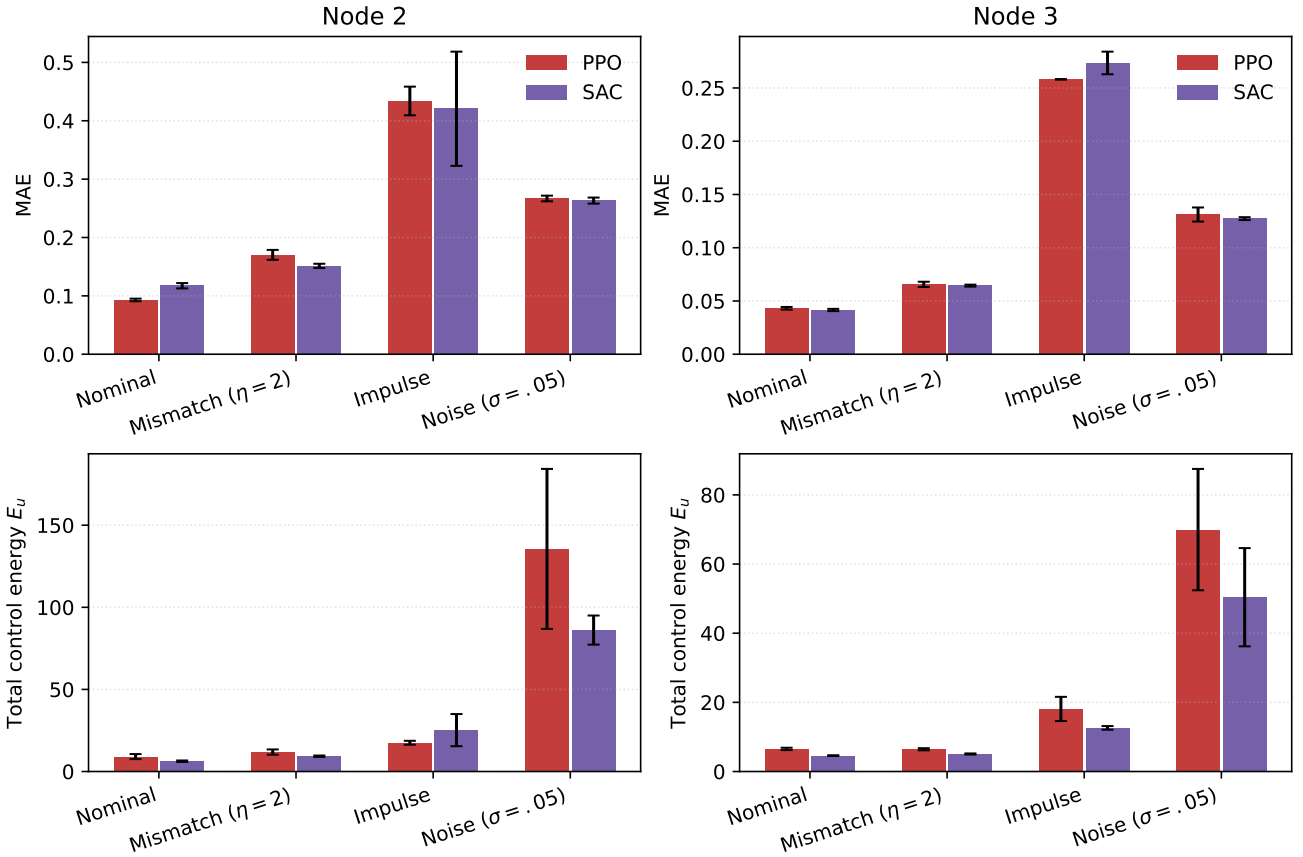


Figure 11: PPO–SAC comparison at matched environment-transition budgets. Bars show averages across training-run means; error bars indicate their sample standard deviation. The upper panels use full-horizon MAE for every scenario; the separate post-impulse metric is given in Table 13.

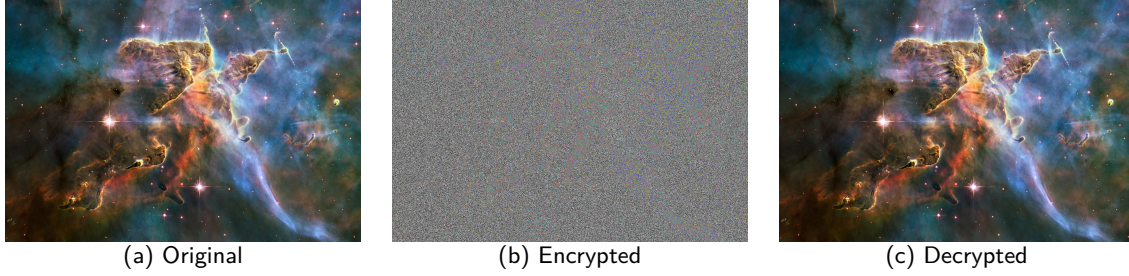


Figure 12: The original, encrypted, and decrypted images.

5. Image Encryption Scheme and Statistical Security Analysis

The legacy implementation and its existing image statistics are described first. Section 5.2 then introduces a separate, conditionally justified reconstruction experiment to examine the missing link between continuous synchronization error and discrete rule agreement; its results must not be attributed to the legacy preprocessing.

5.1. Legacy Image-Transform Implementation

Astronomical systems generate large amounts of high-value image data, including celestial structures, planetary surfaces, and remote-sensing information. In distributed observation scenarios, such as Very Long Baseline Interferometry [42], images may be transmitted between observation sites. This creates exposure to eavesdropping, tampering, and unauthorized access, making secure image protection important for astronomical and space-related applications.

Astronomical images usually contain rich textures, complex intensity distributions, and strong spatial correlations among adjacent pixels, making them suitable test objects for image encryption algorithms. An effective encryption scheme should conceal the visual content of the original image and reduce its statistical redundancy. Since chaotic systems are sensitive to initial conditions and can generate pseudo-random trajectories, they have been widely used for image encryption. In a drive-response synchronization framework, synchronized chaotic trajectories can provide consistent key streams for encryption and decryption.

Motivated by this idea, this section applies synchronized trajectories from the PPO-controlled continuous-time Hopfield chaotic neural network to image encryption. Figure 13 shows the workflow. The scheme combines synchronized Hopfield chaotic sequences, Logistic-map-based masking, and dynamic DNA coding. The plain image is encrypted and then decrypted to verify reversibility. RGB histograms and adjacent-pixel correlations are further analyzed to evaluate the statistical properties of the encrypted image. The specific steps are as follows:

1. *Channel separation and spatial blocks.* Let $P \in \{0, \dots, 255\}^{H \times W \times 3}$ be a three-channel image. Each channel is sliced into non-overlapping blocks of edge length $p = 8$, visited from left to right and then from top to bottom. Boundary blocks may be smaller. Thus,

the number of blocks is

$$N_b = \lceil H/p \rceil \lceil W/p \rceil. \quad (52)$$

Write $B_{c,b} = \mathcal{F}_p(P_c)_b$, where $\mathcal{F}_p$ denotes this ordered slicing operation, not a permutation of pixels. Its inverse places each block back at its original location.

2. *Image-dependent Logistic mask.* The implemented image feature is the sum of all channel values,

$$S(P) = \sum_{c=1}^3 \sum_{i=1}^H \sum_{j=1}^W P(i, j, c). \quad (53)$$

Let d be the decimal string representing $S(P)$. The password string is concatenated directly with d , encoded in UTF-8, and hashed using SHA-256. Interpreting the digest as a nonnegative integer h , the implementation initializes a scalar floating-point value $z_0 = h/2^{256}$. The endpoint values zero and one, if obtained after floating-point conversion, are replaced by two const float, respectively. The sequence is then generated by

$$z_{t+1} = 3.9999z_t(1-z_t), \quad t = 0, \dots, HW-2. \quad (54)$$

With rnd denoting rounding to nearest with ties to even, as in NumPy, the mask is

$$Q = \text{reshape}_{H \times W}((\text{rnd}(10^4 z_t) \bmod 256)_{t=0}^{HW-1}). \quad (55)$$

The reshape uses row-major order. This is a floating-point mask generator; the hash length is not a claim about its effective key space. The receiver needs the same password and d to reconstruct Q .

3. *Channelwise masking.* Each channel uses the same mask, partitioned by the same operation:

$$D_{c,b} = B_{c,b} \oplus \mathcal{F}_p(Q)_b. \quad (56)$$

Here $\oplus$ is bitwise XOR. The implementation does not contain a circular or sequential cross-channel diffusion stage.

4. *Rule-stream construction.* After the trajectory transient, one sample is collected for each block. From the three drive states, four scalar streams are formed as

$$h_1 = x_1, \quad h_2 = x_2, \quad h_3 = x_3, \quad h_4 = x_1 x_2 + x_3.$$

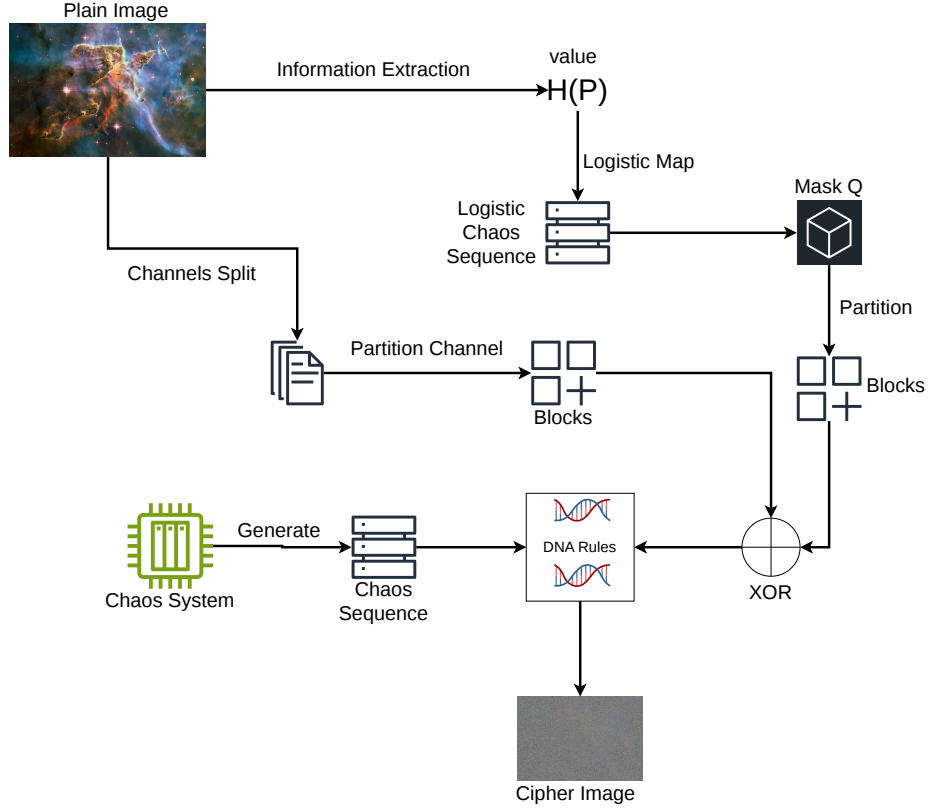


Figure 13: Overview of the image-encryption workflow. The image-derived value denoted by $H(P)$ in the schematic is combined with the password and mapped through SHA-256 to the Logistic initial value, as specified in the text. Chaotic sequences determine the discrete DNA rules after preprocessing. This overview omits the round-level operations and receiver-side reconstruction; channelwise XOR masking is not cross-channel diffusion.

(57)

The fourth stream is derived algebraically, not an additional dynamical state. Response streams use the corresponding y states. For each stream, the implemented preprocessing is

$$q^{(0)} = (\text{rnd}(h) \bmod 8) + 1, \quad (58)$$

$$q^{(\ell+1)} = \lfloor (\mathcal{K}[q^{(\ell)}] \bmod 8) + 1 \rfloor. \quad (59)$$

The second update is repeated for $\ell = 0, \dots, 14$. Here $\mathcal{K}$ is the scalar Kalman filter implemented as a forward pass, with measurement variance 10^{-2} and process variance 10^{-5} . Each pass is restarted with the first input element as its estimate and measurement variance as its initial error variance. The floor represents the implemented conversion of positive values to unsigned integers. The resulting indices are $s_{i,b} = q_{i,b}^{(15)} \in \{1, \dots, 8\}$. This legacy preprocessing is not a dead-zone quantizer and does not provide a bound guaranteeing identical indices from small continuous-state errors.

5. *Two-round DNA substitution and reconstruction.* Let R_i be the bijection in Table 14, applied to each consecutive two-bit pair of a byte, starting from the most

Table 14

DNA coding rules used in the implementation.

Rule	00	01	10	11
1	A	G	C	T
2	A	C	G	T
3	T	G	C	A
4	T	C	G	A
5	C	A	T	G
6	G	A	T	C
7	C	T	A	G
8	G	T	A	C

significant pair. The same block index is used for all pixels of that block and all three channels:

$$D'_{c,b} = R_{s_{2,b}}^{-1}(R_{s_{1,b}}(D_{c,b})), \quad (60)$$

$$D''_{c,b} = R_{s_{4,b}}^{-1}(R_{s_{3,b}}(D'_{c,b})), \quad (61)$$

$$C_c = F_p^{-1}((D''_{c,b})_{b=1}^{N_b}). \quad (62)$$

These operations are pointwise substitutions, not spatial scrambling or propagation between pixels.

For decryption, the response indices are applied in reverse order: encode with the fourth rule, decode with the

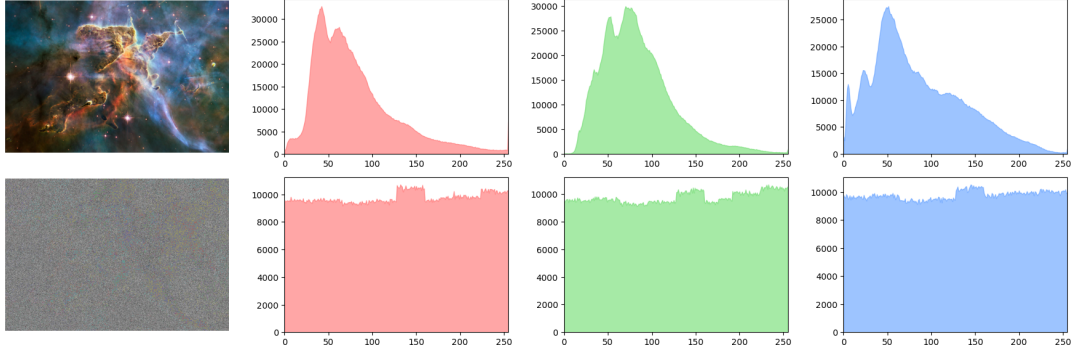


Figure 14: RGB histogram comparison between the plain image and the cipher image.

third, encode with the second, and decode with the first. The result is XORed with the same mask and the blocks are reassembled. Identical drive and response indices, together with identical Q , are sufficient for exact recovery because each DNA map is bijective and XOR is self-inverse. This conditional reversibility does not establish that small synchronization errors yield matching indices, nor that the image transform is cryptographically secure.

The current in-process interface returns both d and the Logistic initial value, but decryption uses only d and recomputes the initial value from the password. This return value is not a specified public communication protocol; in particular, disclosure of the initial value would disclose the mask generator's seed. Reliability of independently generated rule streams and secure exchange of auxiliary information require separate evaluation.

To illustrate the effectiveness of the proposed encryption method, we apply it to a color astronomical image. As shown in Fig. 12(a) and Fig. 12(b), the encrypted image exhibits a noise-like appearance and has no obvious visual similarity to the original image, indicating that the visual information is effectively concealed. Fig. 12(c) shows that the decrypted image is visually consistent with the original image, which illustrates a successful recovery example. Exact recovery is assessed by pixelwise equality rather than visual similarity or the implementation's return flag.

After visual inspection, histogram and adjacent-pixel correlation analyses are used to further evaluate the statistical properties of the cipher image. For each color channel $c \in \{R, G, B\}$, the histogram is defined as

$$H_c(k) = \#\{(i, j) \mid I_c(i, j) = k\}, \quad k = 0, 1, \dots, 255. \quad (63)$$

As shown in Fig. 14, the histograms of the plain image are highly non-uniform, whereas those of the cipher image are much flatter over the gray-level range. This indicates that the encryption process weakens the original pixel-intensity distribution and reduces histogram-based statistical leakage.

The adjacent-pixel correlation coefficient is computed as

$$r_{xy} = \frac{\text{cov}(X, Y)}{\sqrt{D(X)}\sqrt{D(Y)}}, \quad (64)$$

where

$$\text{cov}(X, Y) = \frac{1}{N} \sum_{i=1}^N (x_i - E(X))(y_i - E(Y)), \quad (65)$$

and

$$D(X) = \frac{1}{N} \sum_{i=1}^N (x_i - E(X))^2. \quad (66)$$

Here, X and Y denote two adjacent-pixel sequences sampled in the horizontal, vertical, or diagonal direction. As reported in Table 15, the original images have correlation coefficients close to 1 in all three directions, confirming the strong spatial redundancy of natural images. After encryption, the coefficients produced by both the node2 and node3 models decrease to values close to 0. The scatter plots in Fig. 15 also show that the encrypted pixels are broadly dispersed instead of being concentrated along the diagonal line. These results indicate that the proposed encryption process effectively suppresses adjacent-pixel correlation.

It should be noted that the node1 model is not included because its synchronization performance is insufficient, causing the generated master-slave key streams to fail the sequence consistency check required for correct encryption and decryption. Both node2 and node3 are evaluated under the six-dimensional full-observation setting. Compared with the encrypted images reported in Refs. [43, 44], the proposed method achieves comparable correlation suppression, demonstrating its effectiveness in reducing local statistical redundancy.

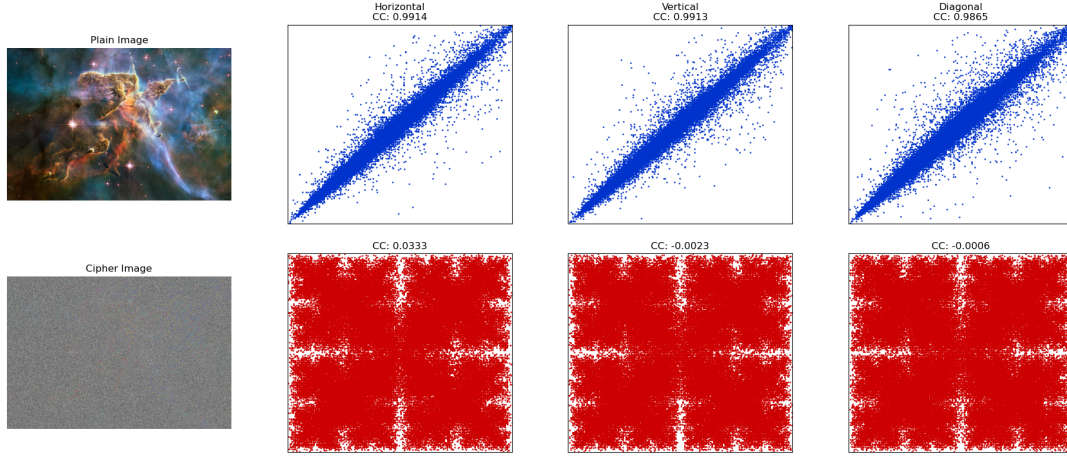


Figure 15: Adjacent-pixel correlation analysis of the plain image and the cipher image in horizontal, vertical, and diagonal directions.

Table 15

Correlation coefficients of the original and encrypted images

Image	Horizontal	Vertical	Diagonal
Original image of node2 model	0.9914	0.9913	0.9865
Encrypted image of node2 model	0.0333	-0.0023	-0.0006
Original image of node3 model	0.9921	0.9917	0.9852
Encrypted image of node3 model	0.0381	-0.0018	-0.0012
Encrypted image of Ref. [43]	-0.0015	0.0023	0.0021
Encrypted image of Ref. [44]	-0.0123	-0.0068	-0.0368

5.2. From Synchronization Error to Discrete Recovery

The control analysis and image application address different requirements. Section 3.3 establishes state boundedness and supplies sampled evidence of local perturbation attenuation, but neither property alone guarantees identical discrete rule streams. For example, rounding 0.49 and 0.51 produces different integers despite their small separation. To make this connection explicit, the following conditional construction is evaluated separately from the preceding legacy image transform and its statistical figures.

Write $h(\mathbf{x}) = (x_1, x_2, x_3, x_1x_2 + x_3)^T$. At each block sample b , the sender uses a fixed width $\Delta > 0$ and forms, componentwise,

$$\begin{aligned} k_{i,b} &= \left\lfloor h_i(\mathbf{x}_b)/\Delta + \frac{1}{2} \right\rfloor, \\ r_{i,b} &= h_i(\mathbf{x}_b)/\Delta - k_{i,b}, \\ s_{i,b} &= (k_{i,b} \bmod 8) + 1. \end{aligned} \quad (67)$$

The receiver receives the aligned centering offsets $r_{i,b}$ as auxiliary information, not the sender's integers, and computes

$$\hat{k}_{i,b} = \left\lfloor h_i(\mathbf{y}_b)/\Delta - r_{i,b} + \frac{1}{2} \right\rfloor, \quad \hat{s}_{i,b} = (\hat{k}_{i,b} \bmod 8) + 1. \quad (68)$$

Proposition 5.1 (Conditional discrete agreement and recovery). *Assume aligned samples, exact arithmetic, unaltered*

auxiliary information, and the same byte mask at both ends. If

$$D = \max_b \|h(\mathbf{y}_b) - h(\mathbf{x}_b)\|_\infty < \Delta/2, \quad (69)$$

then all reconstructed rule indices equal the sender's indices. Any blockwise bijective substitution indexed by these rules is exactly reversible. In particular, if $|x_{j,b}|, |y_{j,b}| \leq M$ for $j = 1, 2$ and $\max_b \|\mathbf{y}_b - \mathbf{x}_b\|_\infty \leq \epsilon$, a sufficient state-error condition is $(2M + 1)\epsilon < \Delta/2$.

Proof. Substituting (67) into (68) gives $\hat{k}_{i,b} = k_{i,b} + \lfloor (h_i(\mathbf{y}_b) - h_i(\mathbf{x}_b))/\Delta + 1/2 \rfloor$. Under (69), the last floor is zero. Thus $\hat{s}_{i,b} = s_{i,b}$; inverse substitution followed by XOR with the identical mask recovers each original byte. For the state-error condition, let $\mathbf{e}_b = \mathbf{y}_b - \mathbf{x}_b$. The fourth-stream difference is $y_{2,b}e_{1,b} + x_{1,b}e_{2,b} + e_{3,b}$, whose magnitude is at most $(2M + 1)\epsilon$; the other three differences are bounded by ϵ . $\square$

State boundedness supplies finite amplitude bounds, but the required uniform ϵ is not proved for PPO in Section 3.3. Accordingly, the application condition is checked on the collected trajectories, not inferred from a negative Lyapunov estimate or a mean error. In finite precision, a positive margin must also cover stream-evaluation, offset-storage, and rounding errors: if the total normalized arithmetic perturbation is bounded by τ , the sufficient condition becomes $D/\Delta + \tau < 1/2$.

Common-pipeline check. The separate prototype replaces repeated filtering with (67)–(68). To prevent serial DNA maps from cancelling, each of the four rule streams selects a permutation on a different two-bit lane of each byte; decryption uses its inverse. The eight permutations are the first eight lexicographically ordered derangements of (0, 1, 2, 3), representing four DNA symbols. Exhaustive checks verified all $8^4 = 4096$ byte maps and their inverses. The original XOR mask is retained only to isolate the functional effect of rule agreement.

PPO and the fixed-gain, LQR, and SMC baselines use the same nominal environment, initial seeds, input limits, and image pipeline, without retraining or retuning. The check uses a 128×128 version of the source image, 8×8 blocks, $\Delta = 1$, and 256 consecutive samples after 1,500 control intervals. Three independently trained PPO policies are each tested from three common initial conditions (nine trials per node); each deterministic baseline uses the same three initial conditions (three trials). No state guard was activated. Direct rounding and reconciliation use the same sender rules, substitutions, masks, and trajectories; only receiver rule reconstruction changes.

Table 16

Functional connection check, separate from the legacy image statistics. $D_{\max}$ is the largest four-stream error across trials. Mismatch is the mean rule mismatch percentage; recovery counts denote exact pixelwise recovery. D/R denotes direct rounding/reconciliation.

Node	Method	$D_{\max}$	Mismatch (%)	Recovery
			D/R	D/R
2	PPO	0.1265	1.563/0	0/9; 9/9
2	Fixed	8.5900	73.307/70.671	0/3; 0/3
2	LQR	0	0/0	3/3; 3/3
2	SMC	0	0/0	3/3; 3/3
3	PPO	0.0360	0.347/0	0/9; 9/9
3	Fixed	0	0/0	3/3; 3/3
3	LQR	0	0/0	3/3; 3/3
3	SMC	0	0/0	3/3; 3/3

Table 16 shows that the tested PPO residuals satisfy the four-stream condition, yet direct rounding still creates mismatched rules. Reconciliation removes these mismatches and gives zero image error in all 18 PPO trials. The same mechanism also works for the successful traditional controllers; the displayed zero stream errors mean identical stored floating-point samples, not a mathematical finite-time synchronization theorem. Node 2 fixed-gain feedback fails the condition and does not recover the image in these tests. These results connect measured synchronization quality to discrete recovery without establishing a PPO-specific encryption advantage or a population reliability rate.

Scope and communication cost. This construction requires aligned auxiliary offsets; their compact JSON representation is approximately 20.7 kB per tested image, about 42% of the uncompressed image bytes. Their information

leakage has not been bounded. The rule digest used to reject disagreements is unkeyed and is not authentication. Passing it is not proof of state synchronization: modulo-eight aliases can preserve the rules even outside (69). Correct recovery is therefore distinct from secrecy, tamper resistance, and secure key agreement. The conditional proposition does not validate the legacy preprocessing or establish cryptographic security for either image transform.

6. Conclusion

This study examined single-input synchronization as a joint choice of actuator location and feedback information. In the specified Hopfield network, equal observation dimensions can lead to markedly different results because the retained coordinates interact with the actuated node. The complete paired-subset PPO comparison identifies this dependence, and the targeted SAC check reproduces the direction of the tested error changes. The finding supports a concrete design principle: choose observation content for the available actuator, rather than treating reduced input dimension as a sufficient description of the feedback configuration.

The actuator-subset and controller comparisons clarify the cost of that choice. Single-input Node 3 achieves low residuals with fewer control channels and less total effort than the favorable multi-input configurations, while those configurations improve full-horizon accuracy. LQR, SMC, and SAC further show that the balance between error and effort depends on controller design and operating conditions. The screening example brings these considerations together by selecting within a fixed candidate pool according to an accuracy requirement and a stated resource priority.

Analytically, bounded actuation yields bounded drive–response trajectories. Finite-time conditional Lyapunov estimates and multi-step checks complement this result by characterizing local perturbation attenuation. Extending the configuration comparisons to other network parameters and developing certified synchronization-error bounds are natural next steps.

A separate image-reconstruction experiment makes the application requirement explicit: a sufficient four-stream error condition, together with aligned auxiliary offsets, guarantees discrete rule agreement and hence exact recovery under an identical mask. The common-pipeline tests verify this connection for the evaluated trajectories, rather than assuming that a small average synchronization error suffices. This functional result does not establish cryptographic security. A final communication protocol requires additional treatment of auxiliary-information leakage, authentication, and key management.

Declaration of competing interest

The authors declare that there is no conflict of interest regarding the publication of this paper.

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

- [1] L. M. Pecora, T. L. Carroll, Synchronization in chaotic systems, *Physical Review Letters* 64 (8) (1990) 821–824.
- [2] S. Boccaletti, J. Kurths, G. Osipov, D. Valladares, C. Zhou, The synchronization of chaotic systems, *Physics Reports* 366 (1-2) (2002) 1–101.
- [3] A. Koronovskii, O. Moskalenko, A. Hramov, On the use of chaotic synchronization for secure communication, *Physics-Uspekhi* 52 (2010) 1213.
- [4] G. Wen, X. Yu, W. Yu, J. Lu, Coordination and control of complex network systems with switching topologies: a survey, *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 51 (10) (2021) 6342–6357.
- [5] M. Yassen, Controlling chaos and synchronization for new chaotic system using linear feedback control, *Chaos, Solitons & Fractals* 26 (3) (2005) 913–920.
- [6] Y. Wang, Z.-H. Guan, H. O. Wang, Feedback and adaptive control for the synchronization of chen system via a single variable, *Physics Letters A* 312 (1) (2003) 34–40.
- [7] M. Feki, Sliding mode control and synchronization of chaotic systems with parametric uncertainties, *Chaos, Solitons & Fractals* 41 (3) (2009) 1390–1400.
- [8] H. Su, L. Runzi, J. Fu, M. Huang, Fixed time stability of a class of chaotic systems with disturbances by using sliding mode control, *ISA Transactions* 118 (2021) 75–82.
- [9] G. Chen, Pinning control and synchronization on complex dynamical networks, *International Journal of Control, Automation and Systems* 12 (2) (2014) 221–230.
- [10] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd Edition, MIT Press, 2018.
- [11] M. A. Bucci, O. Semeraro, A. Allauzen, G. Wisniewski, L. Cordier, L. Mathelin, Control of chaotic systems by deep reinforcement learning, *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 475 (2231) (2019) 20190351.
- [12] I. Carlucho, M. De Paula, G. G. Acosta, An adaptive deep reinforcement learning approach for MIMO PID control of mobile robots, *ISA Transactions* 102 (2020) 280–294.
- [13] S. Vashishtha, S. Verma, Restoring chaos using deep reinforcement learning, *Chaos: An Interdisciplinary Journal of Nonlinear Science* 30 (2020) 031102.
- [14] Y. Y. Han, J. P. Ding, L. Du, Y. M. Lei, Control and anti-control of chaos based on the moving largest lyapunov exponent using reinforcement learning, *Physica D: Nonlinear Phenomena* 428 (2021) 133068.
- [15] H. Cheng, H. Li, Q. Dai, J. Yang, A deep reinforcement learning method to control chaos synchronization between two identical chaotic systems, *Chaos, Solitons & Fractals* 174 (2023) 113809.
- [16] H. Cheng, H. Li, J. Liang, Q. Dai, J. Yang, Generalized synchronization between two distinct chaotic systems through deep reinforcement learning, *Chaos, Solitons & Fractals* 199 (2025) 116727.
- [17] H.-T. Yau, P.-H. Kuo, P.-C. Luan, Y.-R. Tseng, Proximal policy optimization-based controller for chaotic systems, *International Journal of Robust and Nonlinear Control* 34 (2024) 586–601.
- [18] L. Xu, R. Ma, J. Wu, P. Rao, Proximal policy optimization approach to stabilize the chaotic food web system, *Chaos, Solitons & Fractals* 192 (2025) 116033.
- [19] S. Jin, J. Chen, J. Wu, X. Wang, X. Luan, J. Fu, G. Wen, A deep reinforcement learning approach for synchronization between two memristor chaotic systems and application for image encryption, *IEEE Transactions on Circuits and Systems I: Regular Papers* 73 (2) (2026) 1380–1393.
- [20] M. Weissenbacher, A. Borovkyh, G. Rigas, Reinforcement learning of chaotic systems control in partially observable environments, *Flow, Turbulence and Combustion* 115 (3) (2025) 1357–1378.
- [21] D. E. Ozan, A. Nóvoa, G. Rigas, L. Magri, Data-assimilated model-informed reinforcement learning, *Proceedings of the Royal Society A* 481 (2327) (2025) 20250476.
- [22] P. Tooranjipour, R. Vatankhah, Adaptive critic-based quaternion neuro-fuzzy controller design with application to chaos control, *Applied Soft Computing* 70 (2018) 622–632.
- [23] M.-R. Akbarzadeh-T., S. A. Hosseini, M.-B. Naghibi-Sistani, Stable indirect adaptive interval type-2 fuzzy sliding-based control and synchronization of two different chaotic systems, *Applied Soft Computing* 55 (2017) 576–587.
- [24] J. K. Viswanadhapalli, V. K. Elumalai, Shivram S., S. Shah, D. Mahajan, Deep reinforcement learning with reward shaping for tracking control and vibration suppression of flexible link manipulator, *Applied Soft Computing* 152 (2024) 110756.
- [25] J. J. Hopfield, Neurons with graded response have collective computational properties like those of two-state neurons., *Proceedings of the National Academy of Sciences* 81 (10) (1984) 3088–3092.
- [26] J. Li, F. Liu, Z.-H. Guan, T. Li, A new chaotic hopfield neural network and its synthesis via parameter switchings, *Neurocomputing* 117 (2013) 33–39.
- [27] P. Zheng, W. Tang, J. Zhang, Some novel double-scroll chaotic attractors in Hopfield networks, *Neurocomputing* 73 (10) (2010) 2280–2285.
- [28] B. Bao, H. Qian, J. Wang, Q. Xu, M. Chen, H. Wu, Y. Yu, Numerical analyses and experimental validations of coexisting multiple attractors in Hopfield neural network, *Nonlinear Dynamics* 90 (4) (2017) 2359–2369.
- [29] E. E. Mahmoud, L. S. Jahanzaib, P. Trikha, O. A. Almaghrabi, Analysis and control of the fractional chaotic Hopfield neural network, *Advances in Difference Equations* 2021 (1) (2021) 126.
- [30] W. Zhang, J. Huang, P. Wei, Weak synchronization of chaotic neural networks with parameter mismatch via periodically intermittent control, *Applied Mathematical Modelling* 35 (2) (2011) 612–620.
- [31] Z. Cao, C. Li, M.-F. Leung, The synchronisation problem of chaotic neural networks based on saturation impulsive control and intermittent control, *Mathematics* 12 (1) (2024) 151.
- [32] X. Wu, S. Liu, H. Wang, Y. Wang, Stability and pinning synchronization of delayed memristive neural networks with fractional-order and reaction–diffusion terms, *ISA Transactions* 136 (2023) 114–125.
- [33] Z. Dong, J. Shen, C. Lian, S. Wang, Event-triggered global synchronization control of discrete-time delayed neural networks with applications in image encryption, *Journal of the Franklin Institute* (2026) 108524.
- [34] J.-J. E. Slotine, W. Li, *Applied Nonlinear Control*, Prentice Hall, Englewood Cliffs, NJ, 1991.
- [35] L. M. Pecora, T. L. Carroll, Master stability functions for synchronized coupled systems, *Physical review letters* 80 (10) (1998) 2109.
- [36] J. Willems, Least squares stationary optimal control and the algebraic riccati equation, *IEEE Transactions on Automatic Control* 16 (6) (1971) 621–634.
- [37] T. Kailath, *Linear Systems*, Vol. 156, Prentice-Hall Englewood Cliffs, NJ, 1980.
- [38] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms (2017). [arXiv:1707.06347](https://arxiv.org/abs/1707.06347).
- [39] J. Kobiolka, J. Habermann, M. E. Yamakou, Reduced-order adaptive synchronization in a chaotic neural network with parameter mismatch: A dynamical system versus machine learning approach, *Nonlinear Dynamics* 113 (10) (2025) 10989–11008.
- [40] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: *Proceedings of the 35th International Conference on Machine Learning*, Vol. 80 of *Proceedings of Machine Learning Research*, PMLR, 2018, pp. 1861–1870.

- [41] A. Rodríguez-Molina, E. Mezura-Montes, M. G. Villarreal-Cervantes, M. Aldape-Pérez, Multi-objective meta-heuristic optimization in intelligent control: A survey on the controller tuning problem, *Applied Soft Computing* 93 (2020) 106342.
- [42] A. R. Thompson, J. M. Moran, G. W. Swenson, *Interferometry and synthesis in radio astronomy*, Springer, 2017.
- [43] L. Li, A novel chaotic map application in image encryption algorithm, *Expert Systems with Applications* 252 (2024) 124316.
- [44] V. Verma, S. Kumar, Quantum image encryption algorithm based on 3d-bnm chaotic map, *Nonlinear Dynamics* 113 (4) (2025) 3829–3855.